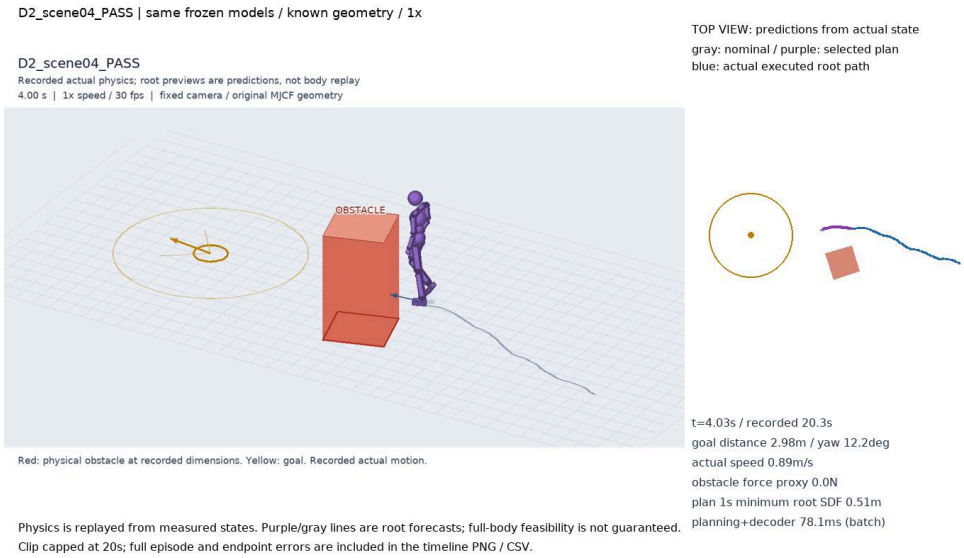
시작

# 내 프로젝트 이해하기

모션 데이터가 실제 로봇 행동이 되기까지

이 문서는 발표 자료가 아니라 **프로젝트를 내 말로 설명하기 위한 학습 노트**다. 각 숫자가 어디에서 왔고, 어떤 코드가 무엇을 학습하며, 최종 결과가 무엇을 뜻하는지 순서대로 읽는다.




최종 시뮬레이션의 실제 실행 장면. 이미지처럼 자세를 강제 재생해 성공을 판정한 것이 아니다.

## 이 프로젝트를 한 문장으로

강화학습으로 만든 휴머노이드의 실제 움직임을 MLP와 Diffusion이 배우고, 실행할 때 장애물 비용으로 미래 움직임을 바꾸어 물리적으로 실행할 수 있는지 비교했다.

> **끝까지 놓치지 않을 세 가지**
>
> ① 모션 예측과 실제 관절 제어는 다르다. ② 최신 실행기는 초기 PPO tracker가 아니라 고정된 학습형 action decoder다. ③ 비용 유도가 도움을 줬지만 모든 장애물·모든 상황을 해결한 것은 아니다.

실험 기준: 2026.11.05 Guidance Comparison V2 · 설명 개정: 2026.11.09. 배경 개념, 계산 예시, 학습과 실행의 연결을 보강한 이해용 보고서다. 예시 숫자는 실험 측정값과 구분하며 새로운 학습이나 평가 결과를 추가하지 않았다.

길잡이

# 읽는 순서

용어를 외우기보다 입력 → 처리 → 출력의 연결을 이해한다.

수식은 **기호의 뜻 → 계산 예시 → 실제 역할 → 코드 위치** 순서로 읽는다. 앞 장의 쉬운 설명과 뒤 장의 구현 세부를 함께 읽으면 같은 개념이 연결된다. 의사코드는 복사 실행용이 아니다.

먼저, 무엇을 해결한 프로젝트인가 04
용어를 사람의 움직임에 대응시키기 05
학습할 때와 실행할 때 06
모션과 action은 다른 것이다 07
RL과 PPO: 점수로 행동을 개선한다 08
PPO 한 번의 학습을 시간순으로 보기 09
DeepMimic과 AMP의 차이 10
Teacher의 모션과 목표는 어떻게 정했나 11
왜 걷기 모션만 재생하지 않았나 12
한 번의 rollout이 학습 자료가 되는 과정 13
데이터를 잘라 학습 표본으로 만든다 14
학습 자료 한 개를 손으로 만들어 보기 15
데이터 수보다 중요한 분리와 품질 16
같은 동작을 같은 숫자로 표현하기 17
숫자의 단위와 좌표를 왜 맞추나 18
660개 입력에서 60×99개 미래 값으로 19
MLP는 무엇을 학습했나 20
MLP가 예측을 배우는 방식 21
Diffusion은 왜 노이즈에서 시작하나 22
Diffusion을 지우개 비유 다음까지 이해하기 23
16번의 복원으로 미래를 만든다 24
실행기는 왜 별도로 필요한가 25

> **읽는 동안 유지할 구분**
>
> Teacher는 경험을 만든다. 생성기는 미래를 제안한다. Decoder는 현재 관절 행동을 만든다. 물리 시뮬레이터는 그 행동의 실제 결과를 계산한다. 예측·실행·성공 판정은 각각 다른 단계다.

길잡이

# 읽는 순서 · 실행과 결과

예측이 실제 움직임이 되는 과정과 결과 해석

수식은 **기호의 뜻 → 계산 예시 → 실제 역할 → 코드 위치** 순서로 읽는다. 앞 장의 쉬운 설명과 뒤 장의 구현 세부를 함께 읽으면 같은 개념이 연결된다. 의사코드는 복사 실행용이 아니다.

최종 decoder를 한 장으로 풀어 읽기 26
실제 한 프레임을 코드로 따라가기 27
왜 예측 2초를 한 번에 실행하지 않나 28
장애물을 모델에 어떻게 전달하나 29
무엇을 나쁜 미래라고 판단했나 30
장애물 정보는 신경망 어디로 들어가나 31
비용이 Diffusion을 어떻게 바꾸나 32
Gradient는 어떻게 “왼쪽으로 피하라”가 되나 33
MLP도 같은 비용으로 수정한다 34
“같은 조건”의 의미와 남아 있는 차이 35
코드 지도: 어느 파일을 읽어야 하나 36
학습 코드와 checkpoint 읽는 법 37
어떤 파일을 어떤 순서로 읽을까 38
성공률을 어떻게 읽어야 하나 39
한 episode가 성공으로 기록되는 과정 40
최종 결과와 읽을 수 있는 결론 41
결과 숫자로 어디까지 말할 수 있나 42
영상에서 원인을 나누어 읽기 43
실패를 진단하는 순서와 개발 과정의 의미 44
이해한 내용을 실제 파일로 확인하기 45
어디까지 이해하고 검증한 것인가 46

> **읽는 동안 유지할 구분**
>
> Teacher는 경험을 만든다. 생성기는 미래를 제안한다. Decoder는 현재 관절 행동을 만든다. 물리 시뮬레이터는 그 행동의 실제 결과를 계산한다. 예측·실행·성공 판정은 각각 다른 단계다.

차근차근 이해하기

# 먼저, 무엇을 해결한 프로젝트인가

컴퓨터 화면에서 걷는 모습과, 물리적으로 걷게 만드는 것은 다르다.

애니메이션은 매 순간의 자세를 지정하면 재생할 수 있다. 하지만 물리 시뮬레이션에서는 몸에 중력이 작용하고 발이 바닥을 밀며 균형을 잡아야 한다. 화면에서 그럴듯한 자세를 만들어도 그 자세에 도달하는 힘과 접촉이 맞지 않으면 넘어진다. 이 프로젝트는 이 두 문제를 연결한다.

## 최종적으로 묻는 질문

**이미 배운 보행 경험으로 미래 움직임을 예측한 뒤, 새로운 장애물에 대한 비용을 실행 시 적용하면, 물리적으로 실행 가능한 회피 행동을 만들 수 있는가?** 그리고 같은 정보를 받은 MLP 출력 최적화와 Diffusion 비용 유도는 어떻게 다른가?

예를 들어 5m 앞 목표로 걸어가는데 중간에 작은 상자가 있다고 하자. 생성기는 앞으로 약 2초 동안 어디로 움직일지 숫자로 제안한다. 비용은 상자와 가까운 제안에 높은 점수를 매긴다. 여기서 비용의 높은 점수는 나쁜 뜻이다. 수정된 제안을 실행기가 관절 명령으로 바꾸고, 실제로 움직인 결과를 다시 관찰한다.

| 주어진 것 | 시스템이 해야 하는 것 |
| --- | --- |
| 현재 몸 상태·최근 행동 | 지금 속도와 자세에서 이어지는 미래를 제안 |
| 목표 위치·도착 방향 | 그 방향으로 이동하고 안정적으로 도착 |
| 알려진 장애물의 위치·크기 | 충돌 위험이 낮아지도록 예측을 수정 |
| 물리 시뮬레이터 | 실제 접촉·넘어짐·도착으로 결과 확인 |

> **하지 않은 일을 먼저 구분하기**
>
> 카메라 영상에서 장애물을 인식하거나 미지의 지도를 만드는 연구가 아니다. 장애물 정보는 이미 알려져 있다. 실제 로봇 실험도 아니며, 단일 평지 시뮬레이터의 제한된 장면에서 평가했다.

차근차근 이해하기

# 용어를 사람의 움직임에 대응시키기

같은 “모델”이라는 말로 불리는 부품들을 역할로 구분한다.

| 용어 | 쉬운 뜻 | 여기서 하는 일 |
| --- | --- | --- |
| Reference | 따라 할 동작의 예시 | 자세·속도 등의 시간 순서 자료 |
| Teacher | 경험을 만들어 주는 정책 | 물리 환경에서 행동해 실제 기록 생성 |
| Policy / 정책 | 상황에 따라 행동을 고르는 함수 | 관측을 관절 action으로 변환 |
| Generator / 생성기 | 미래를 제안하는 모델 | MLP 또는 Diffusion으로 sequence 예측 |
| Decoder / 실행기 | 제안을 행동으로 바꾸는 모델 | 현재 몸과 미래 root로 관절 명령 계산 |
| Root | 몸 전체의 기준이 되는 몸통 좌표 | 위치·방향·이동 및 회전 속도 |
| Rollout | 한동안 실제로 실행한 기록 | 관측·action·상태·종료 등을 저장 |

Teacher와 decoder를 구분하는 이유는 입력이 다르기 때문이다. Teacher는 자신의 목표 조건에 따라 움직여 경험을 만든다. Decoder는 생성기가 제안한 미래 root를 조건으로 받아 그 움직임을 실행한다. 두 모델이 모두 action을 내더라도 같은 가중치나 같은 학습 문제는 아니다.

## 점수도 세 종류다

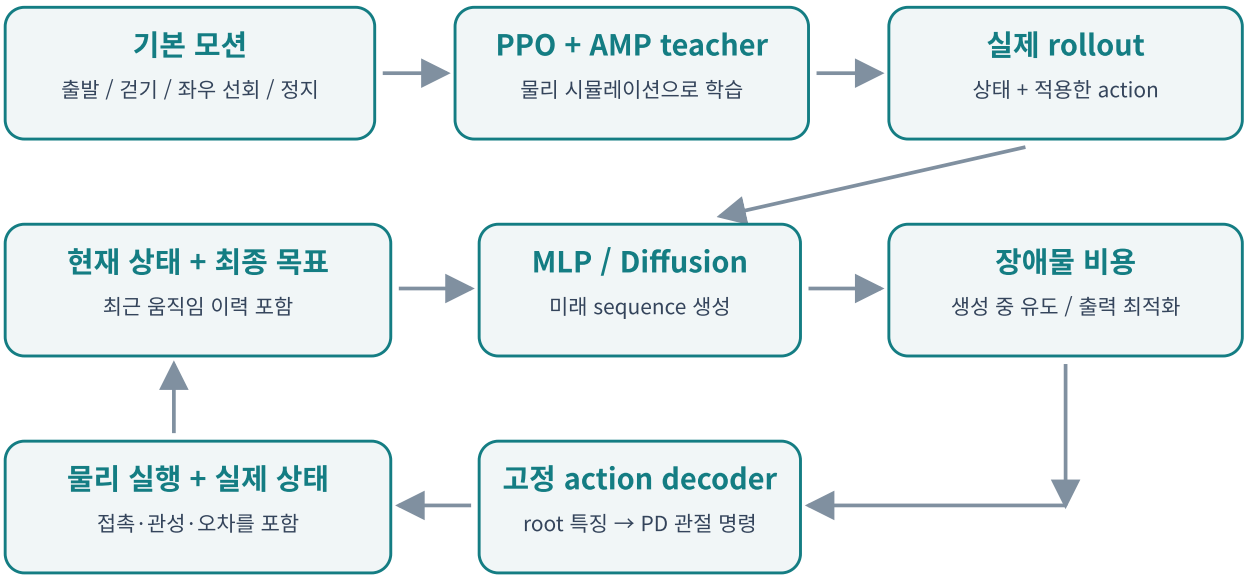
**Reward**는 RL이 한 행동의 결과를 평가한다. 높이고 싶다. **Loss**는 학습 중 예측과 정답 사이의 차이다. 줄이고 싶다. **Cost**는 실행 중 제안된 미래의 충돌 위험 등을 평가한다. 역시 줄이고 싶지만, 보통 모델 가중치를 학습시키는 데 쓰는 것이 아니라 이번 생성 결과를 수정하는 데 쓴다.

> **단어가 같아도 역할은 다르다**
>
> 본 보고서의 prior는 이미 배운 행동이나 움직임의 경향을 뜻한다. 공개 reference, frozen action prior, Diffusion이 배운 모션 분포는 관련되어 있지만 동일한 파일이나 동일한 모델이 아니다.

1부 · 전체 구조

# 학습할 때와 실행할 때

Teacher는 데이터를 만들고, 생성기는 미래를 제안하며, decoder는 실제 관절을 제어한다.




위쪽은 경험을 모으고 모델의 가중치를 바꾸는 **학습 단계**다. 아래쪽은 이미 학습한 가중치를 고정한 채 새 입력에 반응하는 **실행 단계**다. 장애물 비용을 추가하는 것은 실행 중 매번 PPO를 학습한다는 뜻이 아니다.

| 역할 | 무엇을 받나 | 무엇을 내나 |
|---|---|---|
| Teacher | 상태·목표 | 수집용 실제 관절 action |
| 생성기 | 최근 상태·action 이력·목표 | 미래 state-action sequence |
| 실행 decoder | 실제 현재 관측·미래 root 특징 | 실제 적용할 관절 PD 목표 |

**가중치란?**

신경망 안에서 입력을 출력으로 바꾸는 학습 가능한 숫자다. 학습은 이 숫자를 바꾸고, inference는 고정된 숫자로 출력을 계산한다. 비용 유도는 실행 시 생성 중인 값을 바꾸며 신경망 가중치를 다시 학습하지 않는다.

1부 · 물리 제어의 기초

# 모션과 action은 다른 것이다

화면에서 자세를 재생하는 것과, 힘으로 그 자세에 도달하는 것은 다르다.

## 모션 파일은 “어떤 자세였는가”를 기록한다

모션에는 시간에 따른 몸통 위치와 회전, 관절 자세 등이 들어 있다. 화면의 캐릭터 pose를 이 값으로 매 순간 강제 설정하면 물리적으로 불가능한 자세도 그럴듯하게 보일 수 있다. 발이 지면에 닿는 동안 미끄러지거나 몸통이 순간 이동해도, 단순 재생만으로는 드러나지 않을 수 있다.

## 물리 제어는 “어떻게 움직일 것인가”를 실행한다

실제 물리 시뮬레이션에서는 현재 속도와 관성, 중력, 발 접촉, 관절 제한이 다음 상태를 결정한다. Action이 목표 관절 자세를 제시하더라도, 몸이 즉시 그 자세로 바뀌는 것은 아니다. 목표를 향한 관절 구동을 물리 엔진이 적분해 결과를 만든다.

$$\tau \approx K_p(q^{\text{tar}} - q) - K_d\dot{q}$$

τ(토크): 관절을 돌리는 힘의 효과 · q: 실제 각도 · $q^{tar}$: 목표 각도 · $\dot{q}$: 각속도 · $K_p$/K_d: 끌어당김/감쇠 계수. 점 표기는 시간에 따른 변화율이다.

q는 관절 각도, dq는 각속도다. Kp는 목표 자세로 끌어당기는 힘의 크기, Kd는 속도를 감쇠시키는 정도를 뜻한다. 위 식은 제어 원리를 설명하는 근사이며 backend의 실제 drive·force limit·구현 세부는 다를 수 있다.

## 이 프로젝트의 실제 의미

Action은 28개 자유도에 대한 position drive 목표다. 미래 state71이나 한 모션 frame의 길이를 action 차원으로 착각하면 안 된다. Root 위치·방향은 시뮬레이터의 관절 action으로 간접 제어되며, 평가 도중 목표 root 위치로 강제 옮겨 성공시키지 않는다.

> **정지에서 자주 생긴 문제**
>
> 이미 앞으로 움직이는 몸에 고정 자세만 갑자기 요구하면 운동량이 남아 몸이 계속 나아간다. 발 위치가 받쳐 주지 못하면 앞이나 뒤로 넘어질 수 있다. 정지는 속도가 0인 그림 한 장이 아니라, 감속·발 디딤·균형 유지가 연결된 과정이다.

1 · TEACHER

# RL과 PPO: 점수로 행동을 개선한다

Teacher는 정답 action을 외우는 대신 행동의 결과를 경험한다.

**관측 o**는 로봇이 받은 숫자, **행동 a**는 관절 목표, **보상 r**은 결과에 대한 점수다. 정책 π는 관측에서 action을 선택하는 함수다. 상태 s는 시뮬레이터의 실제 물리 상태이고 관측과 완전히 같은 개념은 아니다.

$$o_t \longrightarrow a_t \longrightarrow (r_t, o_{t+1}, d_t)$$

t: 제어 시각 · d: 종료 여부. 이 한 묶음이 transition이다. 여러 step을 실행해 모은 경험을 rollout이라고 한다.

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$$

G: 미래 보상 합 · γ: 먼 미래 점수를 얼마나 반영하는지 정하는 할인율 · T: 종료 시각. 로그의 episode return은 보통 할인하지 않은 보상 합의 평균이며 학습 내내 누적한 총점이 아니다.

PPO는 ① 현재 정책으로 경험 수집 → ② 실제 결과가 예상보다 얼마나 좋았는지 계산 → ③ 좋았던 행동의 확률을 조금 올림을 반복한다. Critic은 기대 점수를 예측하고, advantage A는 그 기대에 비해 얼마나 좋았는지를 나타낸다.

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\text{old}}(a_t \mid o_t)}$$

$$L^{\text{clip}} = \mathbb{E}_t[\min(\rho_t A_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)A_t)]$$

θ: 새 정책 가중치 · ρ: 이전 대비 행동 확률비 · ε: 과도한 업데이트를 억제하는 폭 · E: 표본 평균. 정책은 이 목적을 높이는 쪽으로 바뀐다.

clip은 action 자체를 자르는 것이 아니라 정책 변화의 학습 이득을 제한한다. 현재 설정은 γ=0.99, GAE λ=0.95, ε=0.2, rollout 32 step이다. Iteration은 수집·갱신 한 회전이고 epoch는 모은 데이터를 다시 학습하는 횟수다.

코드 연결 · configs/goal_teacher/agent_sequential_v31.yaml; 외부 MimicKit learning/

차근차근 이해하기

# PPO 한 번의 학습을 시간순으로 보기

걷기 동작 하나를 직접 정답 action으로 넣는 방식과 구분한다.

## 1. 행동하고 기록한다

현재 정책이 관측을 보고 관절 명령을 낸다. 시뮬레이터는 접촉·중력·관성에 따라 다음 몸 상태를 계산한다. 이때 얻은 보상과 종료 여부를 저장한다. 여러 환경을 동시에 돌리는 것은 서로 다른 경험을 병렬로 얻기 위한 것이지, 로봇들이 하나의 몸을 공유한다는 뜻이 아니다.

## 2. 예상보다 좋았는지 계산한다

Critic은 이 상태에서 앞으로 얻을 점수를 예상한다. 실제 결과가 그 기대보다 좋으면 해당 행동을 조금 더 선호하도록 학습한다. 예를 들어 기대 점수가 5인데 실제 추정 결과가 7이라면 “기대보다 좋았다”는 양의 신호가 생긴다. 이 숫자는 개념 예시이며 실제 advantage는 여러 시점의 보상과 가치 추정을 결합한다.

## 3. 정책을 조금 바꾸고 다시 경험한다

Actor는 action을 고르는 정책이다. PPO는 이전 행동의 확률을 너무 급하게 바꾸는 학습 이득을 제한한다. 수집 당시의 old policy는 비교 기준으로 고정하고 새 policy를 갱신한다. 갱신한 정책으로 새 경험을 다시 수집한다. 예전 경험만 무한히 반복하면 현재 정책에서 발생하는 상태를 제대로 반영하지 못할 수 있다.

| 숫자 | 무엇을 세는가 |
| --- | --- |
| Environment step | 환경이 한 번 진행한 것 |
| Samples | 여러 환경에서 모인 transition 수 |
| Iteration | 경험 수집과 학습 갱신의 한 회차 |
| Epoch | 학습 자료를 다시 훑는 횟수 |

> **로그가 좋아도 걸음이 나쁠 수 있다**
>
> 높은 return은 설정한 보상을 잘 얻었다는 뜻이다. 발 끌기·비대칭 보행도 보상 설계에 따라 점수가 높을 수 있다. 실제 영상, 발 접촉과 목표 오차를 함께 확인해야 하는 이유다.

1부 · 모션 사전지식

# DeepMimic과 AMP의 차이

모범 자세를 시각별로 따라갈 것인가, 모션다운 움직임의 분포를 배울 것인가.

## DeepMimic: 시간에 맞춘 모션 추종

DeepMimic은 reference와 실제 캐릭터의 자세·속도·몸통·주요 부위 위치 차이를 이용한 모방 목표를 RL에 결합한다. 초기 reference tracker는 외부 reference target을 관측하고 따라가도록 학습했다. 모션 파일을 화면에서 재생하는 것과 달리, 관절 구동과 물리를 거쳐 자세를 실현해야 한다.

## AMP: 움직임의 스타일을 보상으로

AMP는 실제 시뮬레이션 움직임과 reference 모션의 짧은 관측 이력을 discriminator가 구분하도록 학습한다. Policy는 task 보상과 함께 "reference 데이터의 움직임처럼 보이는가"라는 스타일 보상을 받는다. 모든 순간이 모션의 동일 시각과 일치해야 하는 강한 phase 동기화를 줄일 수 있다.

$$r = 0.65 r_{\text{task}} + 0.35 r_{\text{AMP}}$$

$$r_{\text{AMP}} = -2\log(\max(1 - D(z), 10^{-4}))$$

D(z): 이 움직임이 reference와 닮았다고 판별기가 평가한 값. D가 클수록 AMP 보상이 크다. z는 판별기에 제공한 움직임 특징이다.

여기서 D는 reference 같은 움직임이라고 판단하는 확률이다. AMP reward는 agent에서 더해지므로 환경의 task reward 안에 중복해서 더하지 않는다. Discriminator가 목표 도착까지 판단하는 것은 아니므로 task reward가 별도로 필요하다.

> **현 teacher는 단순한 "보상 하나 추가"가 아니다**
>
> Goal teacher에는 AMP 외에도 목표 진행·전진·회전·정지·넘어짐 종료, 상체와 모션 품질·미끄러짐 등의 항목이 있다. phase-free 가까운 전체 자세/속도 reference를 활용한 품질 보완도 포함한다. 따라서 순수 AMP 하나만으로 모든 동작을 얻었다고 설명하면 안 된다.

1 · TEACHER

# Teacher의 모션과 목표는 어떻게 정했나

기본 동작 다섯 역할이 많은 물리 경험의 출발점이다.

| 모션 역할 | 의미 |
| --- | --- |
| 출발 / 걷기 | 정지에서 걷기 시작 / 지속적인 이동 |
| 좌회전 / 우회전 | 몸과 지지발을 조절하며 방향 전환 |
| 정지 | 감속해 서기. 단순한 고정 자세와 다름 |

Mixamo·MimicKit 기반 자료를 generic humanoid에 맞게 변환하고 발 축·root 이동·jitter를 확인했다. 최종 prior 목록은 정지 유지와 좌우 반전 파생본을 포함한 10항목이다. 독립 원본 모션 10개라는 뜻은 아니다. G1 속도 모델은 이전 이력이며 최종 모델과 다르다.

## 목표로 향하는 방향과 도착 방향은 별개

예를 들어 오른쪽 5m로 이동한 뒤 왼쪽을 바라보라고 할 수 있다. 이동 중에는 목표 위치 쪽으로 향하고, 근처에서는 최종 방향을 맞춘다. 실행 환경의 정렬 진입 조건은 거리 0.24m·속도 0.18m/s 이하, 이동 복귀는 거리 0.6m 초과다. 서로 다른 문턱은 경계에서 상태가 계속 바뀌는 것을 줄인다.

방향 요청은 한 번에 최대 45°로 제한하고 arrival turn_route는 left다. 이런 기존 목표 관측 보조는 모든 비교에 공통으로 남아 있다. 목표 위치를 순간이동시키거나 성공 판정을 대신하는 기능은 아니다.

## 보상의 균형

진행·전진·회전·감속·정지와 AMP·자세 품질·발 미끄러짐을 함께 본다. 속도만 맞추면 발을 끌 수 있고, 원본 모션만 강하게 요구하면 명령을 무시할 수 있다. 뒤로 넘어짐도 벌점 하나보다 감속과 지지발 동작을 같이 봐야 한다.

코드 연결 · configs/datasets/humanoid_five_motion_v1.yaml → 초기 목록; outputs/teacher_symmetric_prior_v13_retry/motions.yaml → 후속 prior; src/humanoid_imitation/envs/goal_amp_env.py → 목표 환경

차근차근 이해하기

# 왜 걷기 모션만 재생하지 않았나

Reference를 주는 일과 새로운 명령에 반응하는 정책을 학습하는 일은 다르다.

고정 reference는 이미 정해진 시간표다. 반면 “지금 왼쪽을 향해라”라는 명령은 현재 속도·지지발·몸의 기울기에 따라 다른 action을 요구한다. 같은 좌회전도 왼발이 땅에 있을 때와 오른발이 땅에 있을 때가 다르다. RL은 이런 실제 상태에서 행동하고 결과를 받는 과정을 거친다.

## DeepMimic 방식과 AMP 방식

Reference tracking 보상은 현재 자세가 해당 시각의 목표 자세에 얼마나 가까운지 비교할 수 있다. AMP는 실제 움직임 특징이 예시 모션의 특징과 비슷한지 판별기로 평가한다. 시간표의 한 자세를 정확히 맞추는 것과, 모션의 성질을 유지하면서 과제를 수행하는 것은 다른 제약이다. 이 프로젝트의 데이터 teacher에는 PPO+AMP를 사용했다.

판별기는 예시 모션과 정책이 만든 움직임을 구분하는 학습을 한다. 정책은 과제 보상을 얻으면서 예시와 닮은 움직임을 만들도록 학습한다. “닮았다”는 점수만으로 관절 한계나 낙상이 사라지는 것은 아니다. 물리 환경과 과제·품질 보상도 필요하다.

## 기본 모션이 다섯 개면 경험도 다섯 개인가?

아니다. 다섯 역할의 prior로 출발하더라도 목표와 초기 상태를 바꾸며 teacher를 여러 번 실행하면 다양한 실제 경험이 생긴다. 그러나 반복·미러링·crop을 많이 했다고 독립적인 원본 동작 종류가 그만큼 늘었다고 말할 수는 없다. 이 구분이 데이터 수와 일반화 주장을 해석할 때 중요하다.

> **이 단계의 성과와 다음 단계의 성과**
>
> Teacher가 잘 걷는 것은 데이터 생성 기반이 좋아졌다는 뜻이다. MLP·Diffusion이 같은 수준으로 실행된다는 증거는 아니다. 생성기가 만든 오차와 decoder의 실행 오차를 별도로 검사해야 한다.

2부 · 실제 데이터

# 한 번의 rollout이 학습 자료가 되는 과정

예측한 모션이 아니라 실제로 적용한 action과 그 결과를 저장한다.

$$(o_t, a_t^{\text{applied}}, s_t, s_{t+1}, d_t, g)$$

Teacher가 action을 내더라도 관절 범위를 넘으면 clamp가 적용될 수 있다. 이 경우 요청한 action과 실제 적용한 action은 다르다. 미래 sequence의 action 정답에는 실제 적용 값을 사용해야 다음 physical state와 맞는 인과관계가 된다.

## 상태 기록의 항목

| 필드 | 뜻 |
|---|---|
| root_pos / root_rot | 실제 몸통의 world 위치 / 회전 quaternion |
| root_vel / root_ang_vel | 실제 선속도 / 각속도 |
| q / dq | 28 DoF 관절 각도 / 각속도 |
| obs | policy가 그 순간 받은 실제 관측 |
| applied_action | 물리 엔진에 보낸 28차원 PD 목표 |
| done / source / split | 종료 여부, 원본 경험의 계보, 학습/검증 구분 |

## 정답이 바뀌면 데이터의 의미도 바뀐다

직접 action 모방의 일부 개발 단계에서는 student가 방문한 상태에 teacher가 제시한 action을 정답으로 붙였다. 하지만 H60 joint sequence 정답은 그 episode에서 **실제 적용된 action과 다음 상태**를 사용한다. Teacher label을 받았다는 것과 teacher가 실제 실행했다는 것은 구분해야 한다.

### Reset 경계

넘어져 reset한 다음 상태를 같은 연속 episode의 미래로 붙이면 로봇이 순간 이동하는 것을 정답으로 배운다. done 이전/이후를 구분하고 state continuity를 검사하며, 미래가 모자라는 window를 임의 padding하지 않는다.

2 · 데이터

# 데이터를 잘라 학습 표본으로 만든다

한 episode에서 여러 window가 나오지만, 서로 독립된 경험은 아니다.

한 번의 목표 수행 기록을 episode라고 한다. 그 안의 시각 t를 잡고 과거 4관측·3action을 입력으로, 이후 60step을 정답으로 자르면 한 window가 된다. 인접 window는 대부분 같은 장면을 공유한다.

| 최종 H60 자료 | 크기 |
|---|---|
| Train / validation windows | 102,593 / 33,638 |
| 그중 새 경험 windows | 45,600 / 19,500 |
| 새 경험 인증 묶음 | train 29 / validation 8 triplet |
| 최종 배치 비율 | 기존 75% + 새 경험 25% |

Triplet은 같은 source에서 갈라진 경험 묶음이다. Source는 원래 경험의 계보다. Crop·mirror·반복 경험을 train과 validation에 나눠 넣으면 거의 같은 정답으로 시험하는 누출이 생긴다. Builder는 ID 겹침을 검사하고, 평균·표준편차는 train 통계만 쓴다.

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \qquad x_i = \sigma_i \tilde{x}_i + \mu_i$$

x: 물리 단위 값 · μ/σ: train 평균/표준편차 · 물결표: 정규화한 값. 서로 단위가 다른 숫자의 학습 규모를 맞추고 실행 전 되돌린다.

## Teacher 성공만으로 충분하지 않은 이유

Teacher가 만든 경로도 고정 실행기가 따라가지 못하면 생성기의 좋은 정답이 실제 성공으로 이어지지 않는다. 새 certified 경험은 실행 재검사를 했지만 모든 legacy 경험이 최신 decoder로 재인증된 것은 아니다. Clip 끝을 정지로 간주하지 않고 실제 저속·저회전 유지 여부를 확인했다.

코드 연결 · state_action_sequence.py: build_windows / valid_starts;
outputs/obstacle_repair_v10/code/build_motion_h60_v10.py

차근차근 이해하기

# 학습 자료 한 개를 손으로 만들어 보기

아래 시각과 위치는 설명용 예시이며 새 실험 결과가 아니다.

어떤 episode에서 로봇이 걷는 중간 시각을 t=3초로 잡자. 30Hz에서 최근 관측 네 개는 대략 2.9초부터 3초까지의 짧은 이력이다. 세 개의 이전 action은 그 관측들 사이에 실제로 무엇을 명령했는지 알려 준다. 이것을 붙인 660개 숫자가 입력 context다.

```
입력: 관측 4 × 144 + 이전 action 3 × 28 = 660
정답: 미래 60 × (action 28 + state 71) = 5,940
한 표본의 미래 길이: 60 / 30 = 약 2초
```

정답은 teacher가 실제로 수행한 다음 움직임에서 온다. 모델은 입력만 받고 이 정답과 비슷한 미래를 내도록 학습한다. Train에는 이런 표본이 많지만, 실행 시에는 정답 미래가 없다. 그래서 model(context)를 호출해 미래를 추정한다.

## 어떤 값은 정답으로 있고, 입력으로는 없어야 한다

미래 실제 발 위치나 관절 각도를 입력에 넣으면 답을 미리 알려 준 셈이다. 목표 위치는 사용자가 알고 주는 조건이므로 입력에 있어도 된다. 실제 미래 endpoint를 목표 label로 쓰는 경우에도 미래 관절 전체까지 입력으로 넘겨서는 안 된다. 미래는 “정답”, 목표는 “달성하고 싶은 조건”이다.

최신 sequence의 action은 앞 8step만 유효하게 감독하고 나머지는 mask로 제외한다. 반면 state는 60step을 학습한다. 따라서 60step action 전체가 모두 같은 수준의 유효 정답으로 학습됐다고 해석하면 안 된다.

> **정답과 실행 입력의 차이**
>
> 학습 표본은 teacher가 잘 수행한 상태에서 오는 경우가 많다. 실행 중 decoder가 조금 밀리면 생성기가 학습에서 거의 보지 못한 이력을 받는다. 이 차이를 distribution shift라고 한다.

차근차근 이해하기

# 데이터 수보다 중요한 분리와 품질

많이 저장했다는 사실만으로 새로운 상황을 잘 배운 것은 아니다.

길이 10초짜리 경험을 1/30초씩 이동하며 잘라 수백 개 window를 만들 수 있다. 이웃 window는 대부분 같은 움직임이다. 하나를 train에, 바로 옆 것을 validation에 넣으면 시험 문제를 거의 미리 본 것과 비슷하다. 원본 경험의 계보를 묶어 나누는 이유다.

## Train·validation·test

Train은 가중치를 바꾸는 자료다. Validation은 학습 중 후보를 고르는 자료다. Test는 선택이 끝난 모델의 결과를 확인하는 자료다. Test 결과를 보고 계속 설정을 고르면 그 자료도 사실상 개발에 사용한 것이 된다. 이번 최종 장면은 개발 장면 재사용이므로 완전히 새로운 시험이라고 주장하지 않는다.

## 성공만 남겨도 해결되지 않는 문제

좋은 teacher 경험은 잘못된 동작을 정답으로 학습할 위험을 줄인다. 하지만 실행 오차로 생긴 낯선 상태에서 어떻게 회복하는지까지 저절로 알려 주지는 않는다. 성공 경험만으로 학습한 모델이 실패 직전 상태를 못 다룰 수 있다. 오차 회복 데이터는 이 문제를 보완할 수 있지만, 이번 문서에서 새로 수집하거나 검증하지 않았다.

## 정지를 제대로 label해야 한다

파일의 마지막 프레임이라는 이유로 정답을 정지로 지정하면, 움직이던 몸을 갑자기 고정하도록 배울 수 있다. 실제 이동속도·회전속도가 작고 균형을 유지하는 구간인지 봐야 한다. 목표 근처를 통과한 것과 도착해서 서 있는 것도 서로 다르다.

코드 연결 · scripts/build_motion_h60_v10.py / planning/state_action_sequence.py → window·source·통계 생성 경로

2부 · 좌표와 단위

# 같은 동작을 같은 숫자로 표현하기

위치·회전·속도의 좌표가 어긋나면 모델보다 입력 변환이 먼저 문제다.

## World와 현재 몸 기준 좌표

World 좌표는 시뮬레이터 바닥에 고정된 좌표다. 몸 기준 좌표는 현재 heading을 정면으로 본다. 같은 앞으로 걷기라도 world에서 북쪽·동쪽을 향하면 숫자가 달라지므로, 미래 state는 현재 root XY를 빼고 현재 yaw의 반대로 회전해 표현한다.

$$\mathbf{p}_{xy}^{\text{local}} = R_z(-\psi_t)(\mathbf{p}^{\text{world}} - \mathbf{p}_t)_{xy}$$

$$\mathbf{v}^{\text{local}} = R_z(-\psi_t)\mathbf{v}^{\text{world}}$$

ψ: 현재 몸의 yaw(수직축 주위 방향) · R_z: 수직축 주위 회전행렬. 예를 들어 동쪽이든 북쪽이든 현재 정면 1m를 같은 상대 좌표로 표현한다.

Root 높이 z는 현재 높이를 빼는 대신 절대 높이를 유지한다. Quaternion은 x,y,z,w 순서를 사용하며, 6D 회전은 회전행렬 첫 두 column을 펼친 것이다. 회전 벡터를 변환할 때 단순히 숫자 순서만 바꾸면 안 된다.

## 왜 회전에 6개 숫자를 쓰나

각도를 직접 쓰면 −180도와180도가 사실상 같은 방향인데 숫자 차이는 크게 보인다. 6D 표현은 이 불연속을 줄이는 데 유리하다. 하지만 신경망이 낸 6개 값이 언제나 정확히 직교하는 단위 벡터는 아니다. 최종 root 특징에 이 표현 일부가 들어가므로 회전·속도 일관성 검사가 중요하다.

## 시간과 단위

| 항목 | 단위·해석 |
|---|---|
| 위치 / 선속도 | m / m/s |
| 관절 / 각속도 | rad / rad/s |
| 목표 방향 보고 | degree로 표시하되 내부 계산은rad |
| Control / physics | 30Hz / 120Hz, 제어당 물리 step4회 |
| 속도 차분 | (다음 위치−이전 위치) / (1/30초) |

World 목표는 실행 중 고정하고 현재 상태만 새로 관측한다. 매 재계획마다 목표를 현재 위치 기준으로 다시 멀리 옮기면 도착하지 않는 문제를 숨기게 된다.

차근차근 이해하기

# 숫자의 단위와 좌표를 왜 맞추나

모델은 “이 숫자는 미터”라는 상식을 자동으로 알지 못한다.

| 표현 | 예시와 의미 |
| --- | --- |
| 위치 m | 0.5는 50cm |
| 각도 rad | π rad는 180°, π/2 rad는 90° |
| 속도 m/s | 1이면 1초 동안 1m 이동하는 속도 |
| 각속도 rad/s | 각도가 얼마나 빠르게 변하는가 |
| Δt | 제어 한 step의 시간: 여기서는 1/30초 |
| q와 dq | 관절 각도와 관절 각속도 |

현재 위치가 world (10, 4)이고 목표가 (15, 4)라면, 몸이 동쪽을 볼 때 목표는 정면 5m다. 같은 몸이 북쪽을 보면 목표는 오른쪽이다. 세계 좌표 숫자만 같아도 몸 입장에서 해야 할 행동은 달라진다. 현재 몸 기준으로 옮기고 회전하는 local 변환이 이 관계를 맞춘다.

## 회전을 숫자로 나타내는 방법

Yaw는 바닥 위에서 바라보는 방향이다. 전체 몸 회전에는 기울어짐도 있어 단일 yaw로 부족하다. 코드에는 quaternion과 6D 회전 표현이 함께 나온다. Quaternion은 회전을 나타내는 네 숫자이며 순서가 중요하다. 6D 표현은 회전행렬의 두 열을 사용한다. 여섯 개의 별도 관절 각도라는 뜻이 아니다.

179°와 −179°는 실제로 2° 차이지만 숫자를 그냥 빼면 358°다. 방향 오차는 이런 경계를 고려해야 한다. 회전 표현·좌우 반전·속도 차분을 잘못 연결하면 신경망을 더 오래 학습해도 좌표 오류를 해결할 수 없다.

> **속도와 위치는 서로 모순되지 않아야 한다**
>
> 30Hz에서 다음 위치가 0.03m 앞으로 바뀌었다면 단순 차분 속도는 약 0.9m/s다. 같은 예측이 속도를 0이라고 내면 일관되지 않다. Consistency 항이 이 모순을 줄인다.

2부 · 입력과 출력 계약

# 660개 입력에서 60×99개 미래 값으로

큰 숫자를 역할별로 쪼개면 구조가 단순해진다.

| 묶음 | 구성 | 차원 |
|---|---|---|
| 현재 관측 한 개 | 물리 관측140 + 목표4 | 144 |
| 최근 관측 | t−3, t−2, t−1, t의 관측 | 144×4 = 576 |
| 과거 action | t−3, t−2, t−1의 실제 PD action | 28×3 = 84 |
| 생성기 context | 관측 이력 + 과거 action | 660 |
| 미래 state 한 개 | root3 + rot6 + v3 + w3 + q28 + dq28 | 71 |
| 미래 token 한 개 | action28 + state71 | 99 |
| 미래 sequence | 60 step의 token | 60×99 = 5,940 |

물리140차원은 높이1 + 몸통회전6 + 선속도3 + 각속도3 + 관절회전84 + DoF속도28 + 주요5부위 상대위치15다. 회전은 heading 기준으로 표현한다. 목표4개는 상대XY와 방향 오차의 sin/cos다. 미래의 실제 관절이나 접촉 정보를 context에 넣어 정답을 누출하지 않는다.

## 과거 action을 넣는 이유

현재 자세가 같아도 직전에 어떤 관절 목표를 내었는지에 따라 관성·접촉·다음 반응이 달라질 수 있다. 짧은 상태 이력과 action 이력은 한 장의 자세만 보는 것보다 움직임 맥락을 제공한다. 초기 history가 부족할 때는 정의된 반복 관측·zero action 또는 저장한 prefix를 사용한다.

> **출력 전체를 실행하지 않는다**
>
> 현재 모델은 state-action sequence를 출력하지만 실제 장애물 실험의 명령은 공통 decoder가 만든다. 생성 action head는 첫8 step만 유효하게 학습하며, 뒤52 step action은 mask된다. 미래 state는60 step 전체를 예측한다. 5,940개가 모두 직접 제어 신호는 아니다.

코드 연결 · state_action_sequence.py: local_state, build_windows; 최종 runner의 context / physical

2부 · MLP

# MLP는 무엇을 학습했나

입력이 주어졌을 때 미래 움직임을 한 번의 forward로 예측한다.

## 회귀 모델의 직관

MLP는 여러 선형 변환과 비선형 함수를 쌓은 신경망이다. 현재 이력과 목표를 숫자 벡터로 넣고, 학습 데이터의 미래 sequence와 가까운 출력을 내도록 가중치를 조정한다. “MLP는 원래 부정확하다”는 명제는 맞지 않는다. 데이터·표현·학습 목표·실행 입력 분포가 맞으면 높은 성능을 낼 수 있다.

## 이 프로젝트의 실제 구조

660차원 context를1024→512 특징으로 변환하고,512 폭의 출력 네트워크가5,940개 값을 예측한다. 부모 direct-action policy를 frozen prior로 둔 anchored 구조이며, 첫 action에 작은 bounded correction이 붙는다. 총 parameter는5,199,184개다. 현재 실행은 action head 대신 state root 특성을 쓰므로 이 prior가 직접 장애물을 피하는 것은 아니다.

## 학습 정답과 손실

$$\mathcal{L} = \mathcal{L}_{\text{state}} + \mathcal{L}_{\text{root}} + 0.05\mathcal{L}_{\text{action}} + 0.01\mathcal{L}_{\text{consistency}}$$

State와 root 손실은 정규화한 예측과 실제 미래 상태의 제곱 오차다. Root를 한 번 더 강조한다. Action 손실은 유효한 첫8 step을 사용하며 첫 action 비중은16/45로 조정했다. Consistency는 물리 단위에서 예측 위치 차분과 예측 선속도가 맞는지 본다.

## 평균화 문제가 생길 수 있는 조건

같은 입력에서 왼쪽·오른쪽 우회 같은 서로 다른 정답이 섞이면 제곱 오차 회귀는 중간 동작을 선호할 수 있다. 그러나 이 현상이 모든 MLP 실패의 원인이라고 단정하면 안 된다. 최신 대조에서는 출력 최적화가 물리적으로 이상한 경로를 만들던 별도 문제부터 발견하고 수정했다.

> **제곱 오차를 읽는 법**
>
> 예측–정답을 제곱해서 평균한다. 오차 부호가 서로 상쇄되지 않고 큰 오차를 더 강하게 벌한다. Root는 state에도 포함되므로 별도 root 항은 몸통 움직임을 추가 강조하는 역할이다.

차근차근 이해하기

# MLP가 예측을 배우는 방식

숫자를 넣으면 숫자가 나오는 큰 함수라고 생각해도 출발할 수 있다.

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

x: 입력 벡터 · W와 b: 학습하는 가중치와 편향 · σ: 비선형 활성 함수 · h: 다음 층의 숫자. 여기서 σ는 정규화 표준편차와 다른 기호 용법이다.

입력 숫자를 가중치로 섞고 활성 함수를 통과시키는 층을 여러 번 쌓는다. 학습 초기에 출력은 정답과 다르다. Loss가 큰 이유를 역전파로 계산하고 optimizer가 가중치를 조금씩 바꾼다. 최종 실행에서는 가중치를 고정하고 입력에 대한 출력을 계산한다.

## 왜 평균적인 동작이 나올 수 있나

비슷한 현재 상태에서 왼발로 시작하는 정답과 오른발로 시작하는 정답이 섞여 있고 입력이 둘을 구분하지 못한다고 하자. 제곱 오차를 줄이는 한 개 출력은 두 정답의 중간으로 갈 수 있다. 두 동작의 평균이 좋은 균형 동작이라는 보장은 없다. 최근 이력이 필요한 이유 중 하나다.

그렇다고 MLP가 원래 부정확하거나 반드시 jitter한다는 뜻은 아니다. 조건과 정답이 일관되고 학습 분포 안이면 좋은 예측을 만들 수 있다. Jitter는 좌표·시간 정렬·재계획 불연속·데이터 부족·실행 오차에서도 생긴다. 모델 종류만 보고 원인을 단정하지 않는다.

### 고정 모델의 출력

같은 가중치와 같은 입력을 주는 일반적인 평가 모드 MLP는 같은 결과를 낸다. 로봇이 실제로 움직이면 다음 입력이 달라져 다음 예측도 달라진다. 이것이 폐루프 실행이다.

2부 · DIFFUSION

# Diffusion은 왜 노이즈에서 시작하나

정답에 섞인 노이즈를 제거하는 연습을 통해 미래 움직임의 분포를 배운다.

## 학습

학습 때는 실제 미래 sequence에 크기가 다른 Gaussian noise를 섞는다. 네트워크는 현재 context와 noisy sequence, noise 단계 k를 받아 원래의 깨끗한 sequence를 예측한다. 이 구현은 noise 자체를 정답으로 삼는 ε-prediction 대신, 깨끗한 출력 x0를 예측한다.

$$\mathbf{x}_k = \sqrt{\bar{\alpha}_k}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_k}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$$

$$\hat{\mathbf{x}}_0 = f_\theta(\mathbf{c}, \mathbf{x}_k, k)$$

$x_0$: 정답 미래 sequence · c: 현재 context · k: 노이즈 단계 · $\bar{\alpha}$: 남긴 원본 신호 비율. 로봇 시각 t와 혼동하지 않도록 이 노트는 노이즈 시각을 k로 쓴다.

100단계 cosine schedule을 사용한다. 최종 학습에서는 배치512개 중 절반의 noise 단계 k를75~99에서 뽑고, 나머지는 전체0~99에서 뽑아 높은 noise 조건을 더 자주 학습했다. 이미지 생성과 같은 원리를 사용하지만 여기서 한 표본은 픽셀이 아니라 움직임의 숫자 sequence다.

## 생성

실행 때는 정답 미래를 알 수 없으므로 Gaussian noise 하나에서 시작한다. Denoising을 반복해 현재 상태·목표와 일치하는 미래를 만든다. 기본 후보 수 K는1이다. 여러 후보를 대량 생성해 외부 선택기로 성공한 것만 고르는 구조는 사용하지 않았다.

## 가능성과 한계

Diffusion은 같은 조건에 가능한 여러 미래를 표현할 수 있는 모델이다. 그렇다고 noise를 바꾸면 필요한 큰 우회가 반드시 나오는 것은 아니다. 실제 한 train context에서 noise8개를 바꾼 진단은1초뒤 횡방향 표준편차 약6.8mm를 보였다. 이 한 사례는 전체 분포의 증명이 아니지만, 다양성을 자동으로 가정하면 안 된다는 근거다.

최종 Diffusion은 context 특징과 noisy sequence·시간 embedding을 결합하며 총8,856,528 parameter다. MLP보다 parameter가 많아 같은 구조 크기의 순수 비교는 아니다.

> **노이즈를 이해하는 법**
>
> Gaussian noise는 0 주변의 작은 난수를 많이, 큰 난수를 드물게 뽑는 정규분포 잡음이다. N(0,I)는 각 성분의 평균 0·분산 1인 표준 잡음이다. 잡음을 넣었다가 제거하는 문제를 반복해서 배운다.

차근차근 이해하기

# Diffusion을 지우개 비유 다음까지 이해하기

잡음에서 복원한다는 말이 정확히 어떤 학습 문제인지 본다.

정답 미래 sequence를 $x_0$라고 하자. 학습할 때 일부러 잡음을 섞어 흐트러진 $x_k$를 만든다. 모델에는 $x_k$와 잡음 단계 k, 그리고 현재 몸·목표 context를 함께 준다. 모델의 과제는 원래 깨끗한 미래 $x_0$를 예측하는 것이다. 이것을 수많은 정답과 잡음 세기로 반복한다.

| 단계 | 알고 있는 것 | 구하려는 것 |
| --- | --- | --- |
| 학습 | 정답 미래·현재 context·일부러 섞은 noise | 그 정답을 복원하는 가중치 |
| 실행 시작 | 현재 context·새 난수 | 가능한 미래의 첫 추정 |
| 실행 반복 | 덜 시끄러운 sequence·context | 더 일관된 미래 |
| 실행 종료 | 복원한 sequence | decoder에 줄 미래 root |

새 잡음으로 시작하면 다른 가능성을 만들 수 있다. 이것이 다중 가능성을 표현할 잠재력이다. 그러나 이 프로젝트는 여러 후보를 뽑아 가장 좋은 것을 고르는 방식이 아니라 기본 후보 K=1이다. 표현할 수 있다는 성질과 실제로 안정적이고 다양한 동작을 얻었다는 증거는 구분해야 한다.

## 100단계 학습, 16단계 실행

학습에는 100개의 잡음 수준을 사용하고, 실행 sampler는 그중 선택된 16단계를 따라 복원한다. 100번 모델을 실행해야만 학습과 맞는 것은 아니다. DDIM은 깨끗한 미래와 남은 잡음을 추정해 다음 선택 수준으로 옮긴다. 이때 모델 가중치를 다시 학습하지 않는다.

> **아무 데이터 없이 새 기술을 발명하는가?**
>
> 아니다. 학습 데이터에서 가능한 움직임의 경향을 배운다. 비용 유도는 그 경향을 이용해 현재 제안을 조절한다. 학습 범위를 크게 벗어나면 물리적으로 이상한 결과가 나올 수 있다.

2부 · DDIM과 모델 학습

# 16번의 복원으로 미래를 만든다

Diffusion 학습의100단계와 실행의16단계는 다른 숫자다.

## DDIM 업데이트의 의미

$$\hat{\boldsymbol{\varepsilon}} = \frac{\mathbf{x}_k - \sqrt{\bar{\alpha}_k}\,\hat{\mathbf{x}}_0}{\sqrt{1-\bar{\alpha}_k}}$$

$$\mathbf{x}_j = \sqrt{\bar{\alpha}_j}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_j}\,\hat{\boldsymbol{\varepsilon}}$$

j는 k보다 노이즈가 적은 다음 선택 단계다. 깨끗한 예측과 추정 노이즈를 더 낮은 노이즈 비율로 다시 섞는다.

실행 sampler는100단계 schedule에서16개의 시점을 선택해 결정론적 DDIM 형태로 이동한다. Noise 초기값과 context가 같으면 같은 출력을 재현할 수 있다. Sampling seed를 바꾸는 것은 새로운 네트워크를 학습하는 것과 다르다.

## 최종 두 생성기의 학습 조건

| 항목 | 확인된 값 |
|---|---|
| 최종 update / seed | 각8,000 / 학습seed0 |
| Batch / optimizer | 512 / Adam, 학습률1e−4→1e−5 cosine |
| Gradient clip / EMA | norm1 / EMA 비율0.005 |
| Train windows | 두 방법 모두102,593 |
| 모니터링 validation | 균형 샘플2,048개 |
| 가중치 초기화 | 부모의 호환 가중치 상속, optimizer 새로 시작 |
| 학습 손실 | state+root+0.05action+0.01consistency |

최종 checkpoint는 validation state·root·consistency 기준으로 선택했다. Diffusion은 denoising validation 기준이 사용되어 sampled reconstruction과 같은 뜻이 아니다. MLP도 같은 코드 경로의 이름을 가진 선택 기준을 기록하지만 forward 자체는 결정론적이다.

### 학습 시간의 해석

최종 H60 추가 학습 구간의 기록은 MLP약46.8초, Diffusion약71.4초다. 이는 이전 teacher 학습·데이터 수집·부모 모델 학습·실패 실험을 포함한 총 프로젝트 비용이 아니다. 가중치 load는 optimizer와 step까지 복원한 resume와 구분한다.

3 · 실행

# 실행기는 왜 별도로 필요한가

숫자로 제안한 미래와 물리적으로 만들 행동 사이를 연결한다.

초기 PPO tracker는 target 자세·속도를 보고 reference를 따라가도록 강화학습했다. 개별 clip을 잘 따라도 전환·정지에서 발 지지와 관성이 맞지 않아 넘어졌다. 이후 state와 action을 함께 생성했지만 root를 수정해도 첫 action이 충분히 바뀌지 않는 연결 문제도 확인했다.

최종 구조는 **미래 root를 명시적으로 읽는 action decoder**다. 이는 초기 PPO tracker의 다른 이름이 아니다. 실제 상태와 원하는 root 특징을 받아 teacher의 적용 action을 지도학습한다. 현재 비교에서는 모든 방법에 같은 decoder를 고정한다.

$$\mathbf{a} = \mathbf{a}_{\text{prior}}(o) + 0.5\tanh\left(\frac{\boldsymbol{\sigma}_a \odot r_\phi(o, F)}{0.5}\right)$$

물리 action 단위의 개념식. Prior: 이미 학습된 기본 행동 · rφ: 보정망 출력 · F: 미래 root 특징 · σₐ: action 정규화 척도 · ⊙: 원소별 곱. 보정은 최대 ±0.5rad이며 실제 코드는 정규화 공간에서 합한 뒤 복원한다.

| 단계 | 실제 값 |
| --- | --- |
| 미래 선택 | 앞 30step 중 index 2, 7, 14, 29 |
| 시각별 특징 | 상대 XY·높이·방향6D 중 XY·선속도XY·각속도Z = 8 |
| 입력 / 출력 | 현재 obs144 + root32 = 176 → 관절28 |
| 보정망 | 176 → 512 → 256 → 28, frozen prior 포함 총 916,792 parameters |

Decoder가 읽는 것은 생성된 미래 관절 전체가 아니라 **root 요약 32개**다. 생성 action head도 최신 실행에는 직접 쓰이지 않는다. 잘못된 미래 root나 학습 밖의 몸 상태까지 안정적으로 실행한다고 보장할 수 없다.

코드 연결 · planning/future_conditioned_action.py: root_features / FutureConditionedAction.forward

차근차근 이해하기

# 최종 decoder를 한 장으로 풀어 읽기

미래 위치를 조금 옮겼다는 사실이 실제 action 변화로 이어져야 한다.

생성기가 “앞으로 가다가 왼쪽으로 비켜라”라는 root를 냈다고 하자. 관절 action이 원래 직진 action 그대로라면 물리 로봇은 여전히 직진할 수 있다. 미래 상태와 action을 함께 출력했다는 것만으로, 수정된 미래가 현재 action을 바꾸는 연결이 보장되지는 않는다.

```
예측 sequence [60,99]
  → 앞 30개 state만 선택 [30,71]
  → 네 시각 × 여덟 root 특징 [32]
  → 현재 관측 [144]와 결합 [176]
  → 보정망 + frozen 행동 prior
  → 관절 목표 [28] → 물리 step
```

네 시각의 index는 2·7·14·29다. 각 시각에서 위치·방향 표현 일부·속도를 꺼낸다. 이때 미래 관절 정답을 그대로 decoder 입력으로 넣는 구조는 아니다. 생성한 전체 state71 중 root 요약만 읽으므로 전신 계획 전체를 그대로 추종한다고 말해서는 안 된다.

## 작은 보정을 더하는 이유

기존 행동 prior가 기본 보행 action을 낸다. 보정망은 현재 상태와 미래 조건에 맞춰 이를 조정한다. tanh로 보정 폭을 제한해 갑자기 매우 큰 관절 명령을 내는 위험을 줄인다. 하지만 제한이 있다고 균형·접촉 안정성이 수학적으로 보장되는 것은 아니다.

이 decoder는 teacher가 적용했던 action을 정답으로 학습한 모델이다. PPO로 reference tracking 보상을 최적화한 초기 tracker와 다르다. 최종 비교에서 decoder 가중치는 고정하고 MLP·Diffusion·비용 수정 방식만 달리한다.

코드 연결 · future_conditioned_action.py: CHANNELS, INDICES, root_features, FutureConditionedAction.forward

3 · 실행

# 실제 한 프레임을 코드로 따라가기

설명용 의사코드다. 아래 순서는 최종 runner와 대응한다.

```
obs = simulator.observe()                        # [B,144]
context = concat(last_4_obs, last_3_actions)       # [B,660]
context = normalize(context, train_stats)
sequence = generator(context)                     # [B,60,99]
sequence = apply_method_cost(sequence, obstacle)  # 방법마다 다름
features = root_features(sequence[:, :30, 28:])   # [B,32]
action = decoder(normalize(obs), features)         # [B,28]
action = denormalize_and_clamp(action)
obs_next = simulator.step(action)
history.append(obs_next, action)                  # 실제 결과 사용
```

B는 동시에 실행하는 환경 수다. `28:`은 앞 action 28개를 제외하고 state71을 꺼내는 Python slice다. `:30`은 2초 예측 중 앞 1초다. 함수 이름 일부는 이해를 위한 축약이며 그대로 실행하는 스크립트는 아니다. 실제 DP는 비용 함수를 denoising 내부로 전달한다. apply_method_cost는 모든 방법이 생성 후 후처리한다는 뜻이 아니다.

## 2초 예측을 한 번에 전부 실행하지 않는다

H=60은 예측 길이, E=1은 재계획 전 실행 길이다. 제어 한 step=1/30초만 실행한 뒤 새 실제 관측으로 다시 생성한다. 넘어지거나 뒤로 밀린 실제 상태도 입력으로 들어가기 때문에 실행 오차가 다음 예측에 영향을 준다. 목표 world 위치는 그대로 유지한다.

> **왜 그냥 정답 미래를 주면 안 되나**
>
> 학습 시 실제 미래는 정답이다. 실행 때는 알 수 없으므로 모델이 예측해야 한다. 저장된 미래를 decoder에 넣는 oracle replay는 실행기를 검사하는 용도이지 공정한 생성기 성능이 아니다.

코드 연결 · scripts/run_guidance_comparison_v2.py: main의 제어 loop, context → physical → plan → feature → action → env.step

차근차근 이해하기

# 왜 예측 2초를 한 번에 실행하지 않나

계획을 세우는 길이와 실제로 믿고 실행하는 길이를 구분한다.

2초 뒤까지 예측해도 발이 조금 미끄러지면 실제 몸은 계획과 달라진다. 그 오차를 무시한 채 2초 내내 같은 계획을 실행하는 것은 open-loop에 가깝다. 실제 상태를 다시 보고 수정하는 것이 closed-loop, 즉 피드백 제어다.

$$H\Delta t = 60/30 = 2\ \text{s}, \qquad E\Delta t = 1/30\ \text{s}$$

H: 예측 step 수 · E: 다시 계획하기 전 실행 step 수. 미래를 2초 보되 약 0.033초만 실행하고 다시 관찰한다.

## 같은 목표, 달라진 상대 위치

World 목표가 (5, 0)에 있으면 로봇이 움직여도 목표는 그대로다. 다만 로봇 기준 상대 목표는 5m 앞에서 4.9m 앞처럼 바뀐다. 매번 목표 자체를 새로 앞으로 옮기는 것이 아니다. 이 차이를 잘못 구현하면 로봇이 영원히 목표를 쫓게 될 수 있다.

## 오차가 되먹임되는 과정

발이 밀림 → 실제 관측이 계획과 달라짐 → 생성기가 새로운 미래를 냄 → decoder가 그 미래를 실행한다. 좋은 경우에는 오차를 줄이고, 나쁜 경우에는 익숙하지 않은 상태에서 예측이 더 나빠져 오차가 커진다. MLP와 decoder가 별개 모델이어도 실행 중에는 이 고리로 영향을 주고받는다.

> **정지 기준과 정지 행동은 별개**
>
> 성공 monitor는 몸이 충분히 안정적인지 판정할 뿐이다. 기준에 들어왔다고 모터를 자동으로 멈추게 하는 학습이 생기지는 않는다. 실제 정지 행동은 생성기·실행기·공통 목표 처리에서 만들어져야 한다.

3부 · 장애물 정보

# 장애물을 모델에 어떻게 전달하나

신경망이 그림을 보고 판단하는 것이 아니라, 알려진 geometry로 비용을 계산한다.

## 입력 조건과 비용 조건을 구분한다

현재 생성기의 context에는 최근 몸 상태·action과 목표가 들어간다. 장애물의 위치·크기를 새로 학습한 관측 채널로 넣은 것은 아니다. 장애물은 시뮬레이터에서 얻은 원형 또는 회전된 상자 geometry로 표현하고, 생성 중 예측 위치가 가까워지는 정도를 비용 함수가 계산한다.

## Signed distance: 얼마나 가까운가

$$d(\mathbf{p}) = \|\mathbf{p} - \mathbf{c}\|_2 - r$$

원형 장애물: c는 중심, r은 반지름. d>0은 바깥, d=0은 경계, d<0은 안쪽. 예: 중심 거리 0.7m, 반지름 0.2m면 표면까지 0.5m다.

원형 장애물의 SDF는 중심까지 거리에서 반지름을 뺀 값이다. 상자는 점을 상자의 회전 좌표로 바꾼 뒤 직사각형 경계까지 거리를 계산한다. SDF를 쓰면 충돌 여부를0/1로만 보는 것보다, 어느 방향으로 떨어져야 비용이 줄어드는지 미분 신호를 얻을 수 있다.

## 안전 여유와 몸통 경로

계획 비용은 root의 평면 위치가 표면에서0.5m 안으로 들어오면 커지도록 했다. 이는 몸통 점 하나만 피하면 팔·다리도 안전하다고 가정하기보다 일정한 여유를 두려는 근사다. 다만 이 여유만으로 전신 충돌을 정확히 보장하지 못해, 실제 평가에서는 신체 부위와 접촉을 별도로 검사한다.

### 계획과 평가의 geometry 역할

계획은 root 경로의 SDF로 비용을 만든다. 평가는 실제 body segment의 보수적 거리와 contact force proxy로 충돌을 판정한다. 계획의0.5m 여유와 평가의0.08m 충돌 문턱은 다른 항목이다. 계획이 장애물을 안 뚫어도 팔·발이 닿을 수 있다.

3부 · 비용 함수

# 무엇을 나쁜 미래라고 판단했나

충돌만 피하려다 멈추거나 비현실적인 움직임을 만들지 않도록 여러 항목을 함께 둔다.

$$J = 10J_{collision} + J_{consistency} + 0.5J_{course}$$

$$+J_{progress} + 3J_{preview} + J_{feasibility}$$

| 항목 | 계산 아이디어 | 필요한 이유 |
| --- | --- | --- |
| Collision | SDF0.5m미만 위험의 제곱 중 가장 큰5개 평균 | 근접·침범을 강하게 억제 |
| Consistency | 위치 차분과 예측 선속도 차이 | 좌표만 순간적으로 옮기는 편법 억제 |
| Course | 몸이 향한 방향과 선속도 방향 불일치 | 옆으로 끌거나 방향과 속도가 따로 노는 경우 억제 |
| Progress | nominal 끝점 대비0.4m이상 진행 손실 | 제자리에서 멈추기만 하는 회피 억제 |
| Preview | 마지막 속도를 최대2초 연장한 예상 위험 | 2초 horizon바깥의 충돌을 조금 일찍 감지 |
| Feasibility | 속도·각속도·가속도 한계 위반 | 비현실적인 급변과 과속 억제 |

모든 항목은 학습된 reward model이 아니라 사람이 정의한 미분 가능한 비용이다. “Diffusion이 목적을 스스로 발견했다”는 표현보다 “비용으로 주어진 목적에 맞게 생성 과정을 조정했다”가 정확하다.

## Preview의 한계

Terminal velocity preview는 현재 예측의 마지막 속도가 일정하다고 가정한 계산이다. 신체 접촉·관절 제약을 시뮬레이션한 실제 미래 rollout이 아니다. 먼 장애물을 조금 일찍 고려하는 데 도움을 줄 수 있지만, 회피 경로를 찾아 주는 전역 planner는 아니다.

V1의 task/settle 후보는 목표 진행·근거리 속도·위험 gate를 추가했으나 train에서 개선되지 않았다. 최신 V2 주 비교는 기존 route-free 비용에 동일 feasibility 항목을 더한 방식이다. 실패한 후보를 선택된 최종 방식처럼 설명하지 않는다.

차근차근 이해하기

# 장애물 정보는 신경망 어디로 들어가나

최종 비용 유도는 장애물 전용 생성기를 새로 학습하는 방식이 아니다.

기본 생성기는 현재 몸·최근 이력·목표 context를 받는다. 최종 실험의 알려진 장애물 geometry는 비용 함수로 전달된다. 비용 함수는 예측 root가 장애물에서 얼마나 가까운지 계산한다. 그 계산이 생성 과정의 숫자를 바꾸도록 연결된다.

```
현재 몸·목표 → 생성기 → 미래 sequence
                            ↓
장애물 위치·크기 → 비용 함수 J
                            ↓ gradient
                 이번 생성 변수 수정
                            ↓
                      고정 decoder
```

## 거리만 멀면 좋은 경로인가?

장애물만 피하려면 목표 반대 방향으로 도망가도 비용이 작아질 수 있다. 그래서 앞으로 진행하는지, 몸 방향과 이동 방향이 맞는지, 위치 변화와 속도가 일치하는지, 지나치게 급하게 움직이는지를 함께 평가한다. 여러 비용 항은 이 상충하는 요구를 조절한다.

Root가 상자 바깥에 있어도 팔·다리·몸통이 부딪힐 수 있다. 그래서 계획 비용의 root 여유와 실제 신체 충돌 판정을 구분한다. 여유 반경은 위험을 줄이는 근사이지 전신의 모든 접촉을 정확히 예측하는 물리 시뮬레이터가 아니다.

> **Preview의 범위**
>
> 미래 끝점에서 같은 속도로 조금 더 움직인다면 위험한지 보는 preview 항이 있다. 이 부분은 실제 관절을 물리적으로 굴려 본 미래가 아니라 간단한 연장 예측이다. 짧은 horizon의 맹점을 줄일 수 있지만 긴 벽 우회를 보장하지 않는다.

3부 · 생성 중 유도

# 비용이 Diffusion을 어떻게 바꾸나

나쁜 결과를 만든 원인을 noisy sequence 쪽으로 거슬러 올라간다.

## Gradient의 직관

비용 J가 커지는 방향을 gradient라고 한다. Gradient의 반대로 조금 움직이면 비용을 낮추는 방향으로 입력이 바뀐다. Diffusion에서는 깨끗한 예측 $\hat{x}_0$에서 비용을 계산하고, 그 비용을 denoiser를 통과해 현재 noisy sequence $x_k$까지 미분한다. 네트워크 가중치는 바꾸지 않는다.

$$\mathbf{g} = \nabla_{\mathbf{x}_k} J(\text{physical}(f_\theta(\mathbf{c}, \mathbf{x}_k, k)))$$

$$\mathbf{x}_k \leftarrow \mathbf{x}_k - s\sqrt{1-\bar{\alpha}_k}\,\frac{\mathbf{g}}{\max(\|\mathbf{g}\|_2, \varepsilon)}$$

∇는 각 숫자를 조금 바꿨을 때 비용이 얼마나 변하는지 모은 gradient다. s=15는 선택된 유도 강도, ε는 0 나눗셈 방지용 작은 값이다. 가중치 θ는 고정하고 noisy sequence만 수정한다.

선택된 V2는 unit-gradient 형태와 guidance scale15를 사용한다. 각 denoising 시각에서 유도 업데이트를1회 수행하고, 수정한 noisy sequence를 다시 denoise한다. 16단계마다 가중치 학습 없이 이 작업을 반복한다.

## 한 장면의 생각 흐름

현재 예상 미래가 원통 쪽으로 가면 SDF 비용이 커진다. 그 비용이 감소하도록 noisy sequence를 바꾸고 복원하면 미래 root 경로와 관련 state가 달라질 수 있다. 고정 decoder는 바뀐 미래 root 특징을 조건으로 관절 명령을 바꾼다. 실제 몸이 옆을 통과하면 그 실제 위치가 다음 생성의 입력이 된다.

## 왜 후처리와 다른가

이 방식은 최종 출력 root를 따로 잘라 옆으로 이동시키지 않는다. 비용을 모델의 복원 과정에 전달해 학습된 생성기의 출력을 바꾼다. 다만 비용이 강하면 학습 분포에서 벗어나거나 일관성이 나빠질 수 있으며, 항상 실행 가능한 결과를 얻는다는 보장은 없다.

### 여전히 필요한 사람의 설계

비용 항목·계수·geometry·실행기·도착 보조는 사람이 정했다. 비용 유도가 연구적으로 의미 있다는 것과 human effort가 사라졌다는 것은 다르다. 이 프로젝트는 후자를 입증하지 않는다.

코드 연결 · guidance_only.py: guided_sample → state_action_sequence.py: sample_sequence → autograd.grad

차근차근 이해하기

# Gradient는 어떻게 “왼쪽으로 피하라”가 되나

비용이 바뀌는 방향을 숫자로 계산한다.

설명용으로 움직임을 x 한 개 숫자로 나타내고, 원하는 값이 1이라서 비용을 $J=(x-1)^2$로 정의하자. 현재 x=3이면 비용은 4다. 이때 gradient는 $2(x-1)=4$다. x에서 gradient의 작은 배수를 빼면 1에 가까워져 비용이 줄어든다.

$$J(x) = (x-1)^2, \quad x = 3, \quad \frac{dJ}{dx} = 4, \quad x_{\text{new}} = 3 - 0.1 \cdot 4 = 2.6$$

이 계산은 직관을 위한 1차원 예시다. 실제 구현은 높은 차원의 noisy sequence gradient를 정규화해 사용한다.

실제 장애물 비용은 좌표→거리→위험 점수로 이어진다. 생성된 좌표는 신경망 출력이고, 그 출력은 noisy sequence의 함수다. 자동미분은 이 연결을 거꾸로 따라가 “잡음 변수의 어느 성분을 조금 바꾸면 비용이 내려가는가”를 계산한다.

## 무엇을 고정하고 무엇을 바꾸나

학습 때는 모델 가중치 θ를 바꾼다. 비용 유도 실행 때는 θ를 고정하고 이번 샘플의 noisy sequence $x_k$를 바꾼다. 모델을 매 프레임 다시 학습하는 것이 아니다. Gradient가 신경망을 통과하더라도 가중치를 갱신하지 않을 수 있다.

## 왜 유도도 실패하나

작은 개선 방향을 따라가는 local 방법은 멀리 돌아야 하는 경로를 쉽게 찾지 못할 수 있다. 오른쪽과 왼쪽이 비슷하거나 양쪽 모두 당장 비용이 높으면 진행이 막힌다. 너무 강한 유도는 학습된 자연스러운 동작에서 벗어나고, 너무 약하면 충돌을 못 피한다. 높은 유도 강도가 항상 좋은 것은 아니다.

4 · 비교

# MLP도 같은 비용으로 수정한다

비교 대상이 무너진 이유와 고친 범위를 함께 이해한다.

MLP는 한 번에 미래를 내므로 Diffusion처럼 노이즈 복원 중 유도할 수 없다. 대신 출력의 root 움직임을 변수로 두고 같은 장애물 비용이 낮아지도록 최적화한다. 처음에는 좌표를 자유롭게 바꾸어 비현실적인 속도가 나왔고 0/24로 실패했다. 이 결과를 MLP의 본질적 한계로 결론내리지 않았다.

$$\mathbf{v}_{i+1} = \Pi_{v_{\max}}(\mathbf{v}_i + \Delta t\, \mathbf{a}_i), \qquad \mathbf{p}_{i+1} = \mathbf{p}_i + \Delta t\, \mathbf{v}_{i+1}$$

Π는 속도 한계 안으로 제한하는 연산이다. 현재 실제 속도에서 시작해 가속도→속도→위치를 적분하므로 좌표만 갑자기 옮기는 편법을 줄인다.

9개 control knot를 보간하고 Adam 32 step으로 조정한다. 속도 2m/s, 회전속도 1.5rad/s, 가속도 6m/s², 각가속도 8rad/s²를 제한한다. yaw도 각속도로 적분하고 기존 회전 표현을 일관되게 돌린다. 기존 높이와 나머지 모션은 보존한다.

| 방법 | 어디를 바꾸나 |
| --- | --- |
| MLP 출력 최적화 | 생성 후 root control을 제약 내에서 최적화 |
| Diffusion 비용 유도 | 복원 과정의 noisy sequence를 gradient로 수정 |
| Analytic root 보정 | 기하 규칙으로 정한 우회 방향에 출력 root를 수정 |

MLP와 DP에는 같은 비용·geometry·실행기를 썼다. 그러나 MLP는 hard bounds, DP는 soft penalty이고 모델 크기·최적화 횟수·시간도 다르다. “같은 정보 아래의 방법 비교”이지 구조만 다른 완전 통제 실험은 아니다. Root 보정은 별도 기준선이며 Diffusion만의 기여가 아니다.

코드 연결 · feasible_root_optimization.py: integrated_root / optimize_feasible_root; obstacle_root_retarget.py → 별도 기준선

차근차근 이해하기

# “같은 조건”의 의미와 남아 있는 차이

공정한 비교는 무엇을 같게 했는지 명시하는 데서 시작한다.

장애물 정보를 Diffusion에만 주고 MLP에는 주지 않으면 모델 차이인지 정보 차이인지 알 수 없다. 서로 다른 실행기를 쓰면 생성기는 같아도 결과가 달라질 수 있다. 그래서 같은 초기 상태·목표·장애물·비용·고정 decoder·판정으로 실행했다.

| 맞춘 항목 | 그래도 남는 차이 |
| --- | --- |
| 공통 데이터와 목표 정보 | 모델 크기와 학습 목표·checkpoint 선택 방식 |
| 공통 obstacle cost | MLP의 hard bounds와 DP의 soft penalty |
| 공통 물리 실행기 | 각 모델이 제안하는 root의 실행 난도 |
| 같은 장면 | DP sampling 2회, MLP 1회 |
| 같은 판정 | 최적화 계산 시간과 반복 방식 |

따라서 이 비교는 **같은 실행 과제에서 두 방법을 각각 구성했을 때의 시스템 결과**다. “Diffusion이라는 신경망 종류만 바꿨더니 이만큼 좋아졌다”는 순수한 인과 실험은 아니다. 그 주장을 하려면 계산 예산·모델 크기·선택 절차 등을 추가로 통제해야 한다.

## Baseline과 ablation

Baseline은 비교 기준 방법이다. Raw MLP·raw DP는 수정 전 상태를 보여 주고, analytic 보정은 학습 생성만의 기여를 과장하지 않게 해 준다. Ablation은 같은 시스템의 한 부품을 제거하거나 바꾸어 그 영향을 보는 실험이다. 새 부품을 여러 개 동시에 바꾸면 어떤 변경이 효과를 냈는지 분리하기 어렵다.

> **시도 횟수와 학습 반복도 다르다**
>
> Sampling seed는 이미 학습된 모델에서 다른 잡음을 뽑는 것이다. Training seed는 초기화·배치 순서 등부터 다시 학습하는 것이다. Sampling을 두 번 했다고 독립적으로 학습한 모델 두 개를 평가한 것은 아니다.

5 · 코드 읽기

# 코드 지도: 어느 파일을 읽어야 하나

폴더 전체보다 최종 실행 경로를 먼저 읽는다.

| 위치 | 역할과 읽을 부분 |
| --- | --- |
| scripts/run_guidance_comparison_v2.py | 최종 진입점. 모델·환경 로드, 입력 이력, 방법 분기, 물리 실행, 결과 저장 |
| src/humanoid_imitation/envs/goal_amp_env.py | 목표 관측·task 보상. 외부 MimicKit 환경 확장 |
| planning/state_action_sequence.py | local_state: 좌표변환 / build_windows: 학습쌍 / SequenceNet: 기본망 / sample_sequence: DDIM |
| planning/anchored_action_sequence.py | 기본 SequenceNet에 frozen action prior와 유효 출력 mask를 결합 |
| planning/future_conditioned_action.py | root_features와 실제 action decoder |
| planning/obstacle_geometry.py | 장애물 거리, motion_cost, body_clearance |
| planning/guidance_only.py | 유도 sampler 진입점. 선택된 legacy는 sample_sequence로 위임 |
| planning/feasible_root_optimization.py | 제약 root 적분과 MLP 최적화 |
| planning/arrival_success.py | 도착·저속 유지 상태 검사 |
| scripts/analyze_guidance_comparison_v2.py | episode 결과 집계·실패 분류·비교 |

위 표의 `planning/`은 모두 `src/humanoid_imitation/planning/` 아래다. `scripts/`는 실행 절차, `src/`는 재사용 로직, `configs/`는 조건, `outputs/`는 로컬 실험 산출물, `reports/`는 공개 결과다.

> **이름만 보고 최신이라고 판단하지 않기**
>
> 초기 train_motion_mlp.py나 planning/diffusion.py는 과거 reference 모델 경로다. 현재 H60 비교는 state_action_sequence와 anchored 경로다. 기존 파일이 존재한다는 이유만으로 최종 실행에 쓰인다고 보면 안 된다.

5 · 코드 읽기

# 학습 코드와 checkpoint 읽는 법

학습, 저장, 실행은 서로 다른 단계다.

```
# 두 모델의 학습 차이 (의사코드)
context, target = next_batch(dataset)
if method == "mlp":
    pred = model(context)
else:
    k = sample_noise_level()
    noisy = sqrt(alpha[k])*target + sqrt(1-alpha[k])*noise
    pred = model(context, noisy, k)  # clean target 예측
loss = state_loss + root_loss + .05*action_loss + .01*consistency
optimizer.zero_grad()
loss.backward()          # 가중치별 기울기를 계산
clip_grad_norm_(model.parameters(), 1)
optimizer.step()         # 가중치 변경
update_ema()
```

Batch는 한 번에 학습하는 표본 묶음, optimizer는 gradient로 가중치를 바꾸는 규칙이다. EMA는 최근 가중치를 부드럽게 평균한 모델이다. Gradient clip은 갑자기 큰 업데이트가 생기는 것을 제한한다. 실행 시 비용 유도와 달리 여기서는 모델 가중치를 바꾼다.

## 최종 학습 근거 파일

`scripts/train_future_motion_v10.py`가 공개된 H60 학습 코드다. `outputs/obstacle_repair_v10/code/`의 동명 파일은 당시 snapshot, `outputs/obstacle_repair_v10/train_h60_pair.py`는 실행 기록이다. 공개 코드와 로컬 산출물을 구분한다. 최종 runner에는 과거 모듈을 재사용하는 import와 부모 checkpoint 의존성이 남아 있다.

## Checkpoint는 단순한 동작 파일이 아니다

`model.pt`에는 가중치·정규화 통계·구조 정보·부모 경로 등이 들어간다. load는 같은 구조의 모델을 만들고 가중치를 복원하는 일이다. 부모 prior 경로가 필요할 수 있어 model.pt 하나만 복사하면 실행되지 않을 수 있다. Resume는 optimizer와 step까지 복원하는 것이며 이번 추가 학습은 부모 가중치+새 optimizer였다.

코드 연결 · configs/releases/guidance_comparison_v2_manifest.json → 최종 파일·hash;
outputs/obstacle_repair_v10/h60_models/{mlp,diffusion}/result.json → 학습 조건

차근차근 이해하기

# 어떤 파일을 어떤 순서로 읽을까

파일 이름을 외우기보다 값이 이동하는 경로를 따라간다.

## 첫 번째: 실행기의 입구

`scripts/run_guidance_comparison_v2.py`에서 어떤 checkpoint·장면·방법을 읽는지 본다. 다음으로 context를 만드는 부분을 찾는다. 실제 observation과 action history가 어떤 순서로 붙는지 확인한다. 오래된 MLP script를 먼저 보면 최종 구조와 섞여 이해하기 쉽다.

## 두 번째: 생성과 물리 단위 복원

`state_action_sequence.py`의 SequenceNet과 sample_sequence를 읽는다. 학습된 평균·표준편차로 나눈 공간에서 예측하고, 비용·실행 전에 물리 단위로 복원하는 경로를 찾는다. “0.5”가 정규화 값인지 0.5m인지 구분하지 않으면 비용 크기를 해석할 수 없다.

## 세 번째: 미래가 현재 action에 연결되는 곳

`future_conditioned_action.py`의 root_features가 실제로 어떤 채널을 뽑는지 본다. 그 특징이 forward에 들어가 prior action에 어떤 보정을 더하는지 따라간다. 이후 환경 step에 적용되는 action과 저장되는 action이 일치하는지 확인한다.

## 네 번째: 성공 숫자의 출처

`arrival_success.py`는 저속 유지 구간을 관리한다. Runner는 그 판정에 충돌·종료 여부를 결합한다. 분석 script는 episode 결과를 집계한다. 성공률 숫자 하나가 네트워크 자체가 아니라 실행과 판정 전체의 결과임을 알 수 있다.

> **재현의 현실적인 경계**
>
> 공개 코드만 clone하면 모든 가중치·모션이 자동으로 생기는 상태는 아니다. 로컬 outputs의 부모 checkpoint·데이터와 외부 MimicKit 환경이 필요하다. docs/reproduction.md의 공개 범위와 로컬 의존성을 함께 읽어야 한다.

6 · 결과

# 성공률을 어떻게 읽어야 하나

생성된 모양보다 실제 몸의 접촉·도착·정지로 판정한다.

| 항목 | 최종 조건 |
| --- | --- |
| 최대 실행 시간 | 45초 |
| 위치 / 방향 | 목표 1.25m 이내 / 방향 오차 45° 이내 |
| 이동 / 회전 속도 | 0.15m/s 이하 / world 각속도Z 0.2rad/s 이하 |
| 유지 | 연속 5초, 구간 시작·끝 위치 차이 0.1m 이하 |
| 추가 확인 | 이후 2초 낙상 없음 |
| 주 성공 | 위 조건 + 전체 episode 충돌 없음 |

이는 정확히 목표 중심에 멈추는 과제가 아니라 허용 영역에 안정적으로 도착하는 과제다. 5초 drift는 최대 이탈 거리가 아니라 시작·끝 차이다. 추가 2초도 모든 정지 조건을 재검사하는 것이 아니라 낙상 여부를 본다. 판정 한계를 함께 읽어야 한다.

충돌은 비지면 contact force proxy가 1N을 넘거나 실제 신체 segment의 보수적인 표면 거리 하한이 0.08m 미만이면 잡는다. 정확한 mesh 충돌의 수학적 보장은 아니다. 넘어짐은 환경 FAIL, 시간 초과는 별도다.

## 평가 횟수와 독립 상황 수

최신 비교는 24장면·12 source group이다. MLP는 24회, DP는 같은 장면에 sampling seed 2개를 써서 48회다. 학습 seed는 1개다. 목표 거리는 5~6.5m, 목표 위치 방위는 −25°/0°/+25°, 도착 방향은 초기 몸 방향이다. 180° 목표·여러 장애물은 이 최종 평가 범위가 아니다. Train에서 유도 강도와 MLP 학습률 후보를 선택한 후 평가했다. 다만 V1 개발 장면을 재사용했으므로 독립 신규 test-set 일반화라고 말하지 않는다.

코드 연결 · arrival_success.py + V2 runner의 monitors, collision, done; reports/assets/guidance_comparison_v2/protocol.json

차근차근 이해하기

# 한 episode가 성공으로 기록되는 과정

위치에 들어온 순간과 안정적으로 도착한 순간은 다르다.

목표에서 1m 떨어져 있고 방향도 맞더라도 속도가 0.8m/s라면 곧 통과할 수 있다. 최종 판정은 위치·방향뿐 아니라 이동속도와 회전속도도 검사한다. 이 조건을 연속 5초 유지하고 그 구간 시작·끝의 이동이 작아야 한다.

| 설명용 상황 | 판정 해석 |
| --- | --- |
| 목표 옆을 빠르게 통과 | 위치는 들어왔어도 저속 조건 불충족 |
| 2초 서 있다가 다시 움직임 | 5초 연속 유지 불충족 |
| 안정적으로 섰지만 이전에 장애물 접촉 | 충돌 없는 주 성공은 아님 |
| 충돌 없이 도착·유지하고 추가 2초 안 넘어짐 | 나머지 조건도 충족하면 주 성공 |
| 45초 동안 버텼지만 도착 못 함 | 낙상이 아니라 timeout 가능 |

5초 동안의 drift는 시작과 끝 위치 차이다. 중간에 얼마나 크게 벗어났는지의 최대값은 아니다. 다만 그 구간 내 각 시점의 위치·속도 조건도 계속 만족해야 한다. 추가 2초는 낙상 검사를 뜻하며 모든 도착 조건을 새로 2초 유지했다는 뜻은 아니다.

## 허용 범위를 넓히면 무엇이 바뀌나

성공 판정은 쉬워질 수 있지만 실제 몸이 안정적으로 멈추는 능력이 자동으로 좋아지지는 않는다. 모델 비교 도중 한 방법에만 넓은 기준을 주면 공정하지 않다. 최종 보고서의 숫자는 표에 적힌 공통 기준 아래의 결과다.

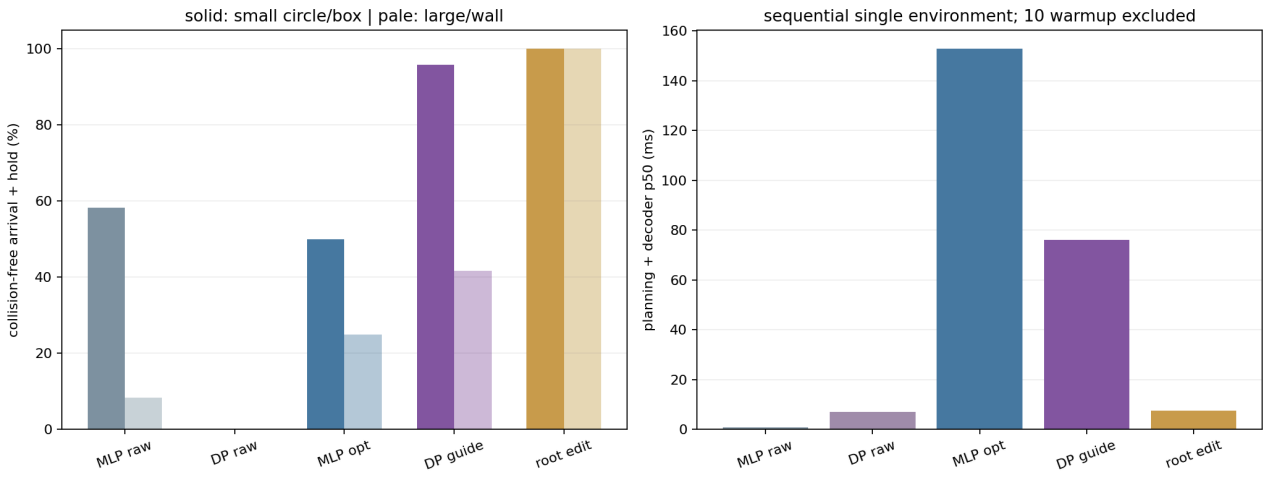
> **논문 수준의 주장으로 바꾸지 않기**
>
> 허용 반경 1.25m·방향 오차 45° 안의 안정 도착을 확인한 것이다. 센티미터 수준의 정밀 위치 제어를 해결했다는 결과는 아니다.

6 · 결과

# 최종 결과와 읽을 수 있는 결론

높은 점수와 남은 실패를 같은 표에서 본다.

| 방법 | 성공 / 낙상 | 계산 p50 |
|---|---|---|
| MLP 원본 | 8/24 · 낙상 8 | 0.9ms |
| MLP 제약 출력 최적화 | 9/24 · 낙상 10 | 152.7ms |
| Diffusion 원본 | 0/48 · 낙상 29 | 7.1ms |
| Diffusion 비용 유도 | 33/48 · 낙상 7 | 76.1ms |
| 별도 analytic root 보정 | 24/24 · 낙상 0 | 7.6ms |




최신 V2 비교. 과거 다른 조건에서 얻은 높은 점수와 합쳐 읽지 않는다.

작은 원형·상자에서는 비용 유도 DP가 23/24, 제약 MLP가 6/12였다. 큰 물체·벽에서는 각각 10/24, 3/12다. DP sampling 2회가 포함되므로 23/24를 독립 장면 24개의 결과로 부르면 안 된다.

이 고정 모델·정보·실행 조건에서 DP 유도가 도움이 됐다. 그러나 raw DP가 장애물에서 0점인 것은 장애물 없는 보행 능력이 0이라는 뜻이 아니다. Root 보정이 가장 강한 결과였으므로 “Diffusion만 해결 가능하다”는 주장도 틀리다.

> **실시간인가?**
>
> 30Hz 제어의 시간 예산은 약 33.3ms다. DP 유도 76.1ms는 physics를 제외한 planning+decoder만으로도 넘는다. 시뮬레이션이 계산을 기다리며 진행한 결과이며 30Hz 실시간 제어를 입증하지 않았다.

차근차근 이해하기

# 결과 숫자로 어디까지 말할 수 있나

분모와 조건을 함께 읽어야 결과가 과장되지 않는다.

비용 유도 Diffusion의 33/48은 약 68.8%이고, 제약 MLP의 9/24는 37.5%다. DP의 48회는 독립 장면 48개가 아니라 같은 24장면에서 두 sampling seed를 실행한 것이다. 각 장면의 난도와 source가 서로 연관되므로 단순한 독립 동전 던지기 48번으로 생각하면 안 된다.

## 강한 증거

고정된 비교 조건에서 비용 유도를 추가한 DP가 raw DP보다 실제 충돌 없는 도착을 많이 만들었다. 작은 원형·상자에서는 23/24로 좋은 결과를 보였다. 이 범위에서 유도의 실용적인 효과가 있다는 것은 결과로 뒷받침된다.

## 제한되는 주장

큰 장애물·벽에서는 10/24로 성능이 떨어진다. 학습 seed도 하나이고 개발 장면을 재사용했다. 다른 로봇·새로운 환경·임의 목표·실제 하드웨어에서도 같은 성공률이 나온다는 증거는 없다. MLP와 DP의 시간·모델 크기 차이도 남는다.

## 가장 높은 analytic 결과를 숨기지 않는 이유

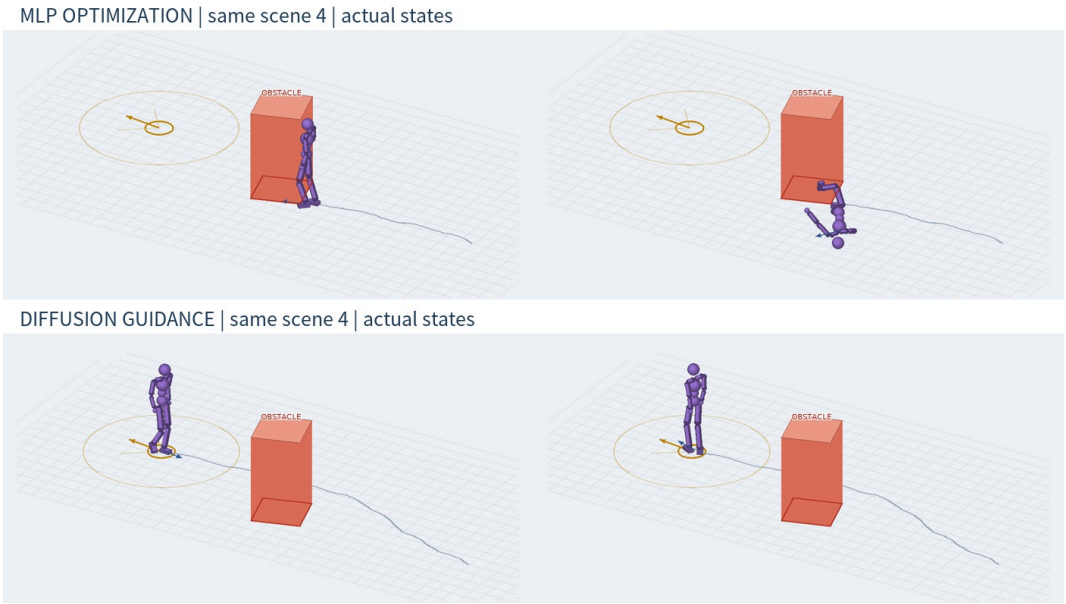
별도 기하학적 root 보정은 24/24였다. 이 결과는 “장애물을 피하려면 반드시 Diffusion이어야 한다”는 주장에 반례가 된다. 연구 질문은 유일한 해결법인가가 아니라, 학습한 움직임을 비용만으로 어느 범위까지 조절할 수 있는가와 그 실패 지점에 있다.

> **Latency를 읽는 예시**
>
> 76.1ms는 0.0761초다. 30Hz에서 한 번의 예산은 33.3ms이므로 이 계산만으로도 예산을 초과한다. 화면이 매끄럽게 재생된 영상과 실시간 처리 속도는 같은 지표가 아니다.

6 · 결과

# 영상에서 원인을 나누어 읽기

예측 root, 실제 몸, 목표를 동시에 비교한다.




같은 상자 장면. MLP 출력 최적화는 접촉 후 낙상, 유도 DP는 성공한 사례다. 한 장면으로 전체 우월성을 판단하지 않는다.

| 관찰 | 우선 점검할 것 |
|---|---|
| 예측부터 장애물을 통과 | 비용·유도·학습 분포·짧은 horizon |
| 예측은 비껴가지만 실제 몸이 접촉 | 전신 여유, decoder 추종, 실제 오차 |
| 도착 후 계속 회전 | 관측 목표 단계, 생성 감속·정렬, 성공 유지 조건 |
| 정지하며 넘어짐 | 잔여 속도·지지발·실행 분포; 정지 pose만 고정해서는 해결 안 됨 |

실패 시 접촉 약 0.5초 전의 예측 root 여유를 확인했다. DP 실패 15개 중 8개는 root 침범, 6개는 여유 부족, 1개는 비접촉 낙상 미해결이었다. MLP 실패 15개는 각각 1·10개, 추가로 안전한 root 대비 실제 접촉 의심 3개·미해결 낙상 1개였다. 이는 원인 후보 분류이지 인과 증명은 아니다.

넓은 벽에서는 큰 우회를 만들지 못하고 벽을 밀다 timeout하는 사례가 남았다. 국소 gradient와 2초 horizon이 한 원인일 수 있지만, 이를 확정하려면 horizon·데이터·실행기를 나눠 추가 실험해야 한다.

차근차근 이해하기

# 실패를 진단하는 순서와 개발 과정의 의미

넘어지는 마지막 프레임보다 그 전에 무엇이 어긋났는지를 본다.

## 1. 정답부터 실행할 수 있는가

저장된 좋은 미래를 같은 실행기에 넣어 본다. 이것도 못 따라가면 생성기를 비난하기 전에 실행기의 입력 계약·학습 범위·접촉 안정성을 점검한다. 이 oracle 검사는 실제 미래를 미리 주기 때문에 모델 비교 점수로 사용하지 않는다.

## 2. 생성된 미래 자체가 모순되는가

Root가 장애물을 통과하는지, 속도와 위치 변화가 맞는지, 갑자기 방향이 바뀌는지 본다. 여기서 문제가 있으면 데이터·학습 loss·비용 유도·재계획 일관성을 확인한다. 실제 몸만 보고는 예측이 잘못된 것인지 알 수 없다.

## 3. 예측은 괜찮은데 실제 몸이 어긋나는가

원하는 root와 실제 root, action 적용 시각, 발 접촉을 맞춰 비교한다. 지연·관성·decoder의 낯선 조건·몸통 대신 발의 충돌 등이 원인 후보가 된다. 몸이 먼저 밀렸다면 다음 생성 결과가 이상해진 것은 원인보다 결과일 수도 있다.

## 이번 프로젝트가 구조를 바꿔 온 이유

초기에는 reference 생성과 PPO tracker를 연결했다. 전환·정지와 오차 회복이 어려워 데이터를 보완했다. 이후 state-action 예측을 시도했지만 미래 root 수정이 현재 action으로 충분히 연결되지 않는 문제가 있었다.
최종적으로 미래 root를 명시적으로 읽는 decoder를 두고, 같은 실행기 아래에서 생성·유도 방법을 비교했다.

> **이력과 최종 결과를 섞지 않기**
>
> 초기의 좋은 tracker 점수, 과거 MLP 성공률, 최신 장애물 점수는 서로 다른 설정에서 나온 결과다. 한 표로 합쳐 최고 숫자만 고르면 전체 시스템 성능을 잘못 설명하게 된다.

마무리

# 이해한 내용을 실제 파일로 확인하기

과거 실험 전체를 외우지 않아도 최종 입력·처리·결과를 추적할 수 있다.

| 확인할 대상 | 위치 |
| --- | --- |
| 최종 MLP / Diffusion | outputs/obstacle_repair_v10/h60_models/mlp/model.pt 및 diffusion/model.pt |
| 공통 실행 decoder | outputs/obstacle_repair_v10/decoder_repair/model.pt |
| H60 학습 자료 | outputs/obstacle_repair_v10/h60_dataset/data.pt |
| 최신 개별 결과 | reports/assets/guidance_comparison_v2/episodes.csv |
| 집계·조건·선택 | 같은 폴더의 summary.json / protocol.json / selection.json |
| 정확한 파일 계보 | configs/releases/guidance_comparison_v2_manifest.json 및 results.json |
| 이전 과정 | docs/development_history.md / docs/experiments/ |

## 핵심 연결을 다시 읽으면

모션 prior → PPO+AMP teacher → 실제 상태·action 저장 → 과거/미래 window → MLP·Diffusion 지도학습 → 같은 목표와 장애물 비용 → 고정 decoder → 물리 실행 → 실제 상태로 다시 계획. 어느 부분에서 문제가 생겼는지 이 연결을 거꾸로 추적한다.

## 참고한 기반

PPO: 정책 업데이트 · DeepMimic: 모션 모방 RL · AMP: 모션 스타일 보상 · DDIM: 적은 단계의 복원 · BeyondMimic: 실행 시 유도로 운동 기술 활용. 본 프로젝트는 이 연구들의 모든 구조·실제 로봇 성능을 재현한 것이 아니다.

PPO·AMP 기반은 외부 MimicKit, 물리는 Isaac Lab/Isaac Sim이다. 목표·데이터·모델 연결·평가를 프로젝트에서 구성했고 구현·분석·문서에는 AI 도움이 포함됐다. 코드 공개와 별도로 모션 라이선스·가중치 재배포 권한은 확인해야 한다.

> **이 프로젝트에서 확인한 범위**
>
> 알려진 단일 정적 장애물의 작은 사례에서 비용 유도 효과를 검증했다. 모델 학습 seed 1개, 개발 장면 재사용, 큰 장애물 실패, 계산 지연과 실행기 간극은 남는다. 독립 반복 평가와 더 나은 실행 가능성 모델링이 후속 과제다.

검토 · 2026.11.06

# 어디까지 이해하고 검증한 것인가

전체 구조의 이해와 알고리즘 숙련, 연구적 증명은 구분한다.

이 노트의 입력·정답·손실·실행 경로와 실패 원인을 내 말로 설명할 수 있다면 프로젝트의 핵심 구조를 이해한 것이다. 그러나 글을 읽고 익숙해진 것만으로 모든 구현을 숙달했다고 볼 수는 없다. 실제 batch의 shape를 추적하고, 한 비용 계수를 바꿨을 때 무엇이 달라지는지 코드와 저장 결과로 연결하는 과정이 필요하다.

## 직접 대조한 세 연구

| 연구 | 공통점과 차이 |
| --- | --- |
| BeyondMimic | 운동 경험과 실행 시 비용 유도라는 큰 방향이 겹친다. 논문은 실제 휴머노이드에서 다양한 운동·waypoint·장애물 회피를 검증했다. 우리는 generic humanoid 시뮬레이션과 별도 root decoder의 제한된 실험이다. |
| CLoSD | 생성과 물리 실행을 실제 상태 피드백으로 연결한다. 논문은 텍스트 조건과 이동·타격·앉기·일어서기 등을 다룬다. 현재 프로젝트는 텍스트 없이 보행·도착과 정적 장애물 하나다. |
| Diffuse-CLoC | 상태·action 공동 모델과 비용 유도를 제어에 이용한다. 현재도 공동 sequence를 내지만 생성 action을 직접 실행하지 않고 root 요약→별도 decoder로 실행한다. 같은 알고리즘을 재현한 것이 아니다. |

BeyondMimic 공식 프로젝트 · CLoSD 공식 프로젝트 · Diffuse-CLoC 논문. 과제·캐릭터·판정이 달라 서로의 성공률을 같은 척도처럼 순위 매길 수 없다.

## 현재 결과의 위치

학부 개인 프로젝트로는 모션 처리·RL·생성 학습·물리 실행·실패 분석을 연결한 깊이가 있다. 연구적 novelty는 아직 제한적이다. 비용 유도나 두 단계 제어 자체는 기존 아이디어이며, 자체 decoder와 실험 설계가 새로운 방법이라는 증거는 별도로 필요하다. 현재 강점은 시스템 구현과 조건을 나눈 비교다.

> **가장 가치 있는 다음 검증**
>
> 구조를 더 붙이기보다 독립 시작 상태·장면을 고정하고 같은 계산 예산에서 비교하는 것이 먼저다. 학습 seed 반복, 공통 decoder 조건과 root 정보 제거 대조는 어느 부분이 기여했는지 밝힌다. 이 항목들은 제안이며 이번 보고서 수정에서 실행하지 않았다.